

CLAUDE.md 완벽 가이드 + 프로젝트 맞춤 설정

1. CLAUDE.md란?

CLAUDE.md는 Claude Code가 모든 대화 시작 시 자동으로 로드하는 마크다운 파일입니다. 기술 스택, 테스트 실행 방법, 중요한 컨벤션, 건드리지 말아야 할 것들에 대한 상시 지시사항을 제공합니다. 이 파일이 없으면 매 세션이 제로에서 시작됩니다. 같은 컨텍스트를 반복 설명하고, 같은 가정을 수정하고, 어제와 똑같은 실수를 반복하게 됩니다.

핵심을 한 줄로:

"새 팀원에게 온보딩할 때 설명하는 모든 것을 Claude에게 한 번만 적어두는 파일"

2. CLAUDE.md 작동 원리

Claude Code의 시스템 프롬프트는 세션 시작 전에 이미 약 50개의 지시사항을 차지합니다. 프론티어 LLM 연구에 따르면 지시사항 따르기는 총 150~200개에서 저하되기 시작하며, 이는 프로젝트에 대해 알려주고 싶은 모든 내용을 위해 약 100~150개의 슬롯이 남는다는 의미입니다. CLAUDE.md가 그 슬롯을 채웁니다.

세션 시작 시 컨텍스트 창 구성:

```
[Claude Code 내장 시스템 프롬프트 ~50개]
+
[CLAUDE.md 전체 내용] ← 매 세션 자동 로드
+
[Memory/MEMORY.md 상위 200줄] ← 자동 학습 내용
+
[현재 대화 내용]
= 컨텍스트 창 사용
```

⚠ 총 150~200개 지시사항이 한계
→ CLAUDE.md는 간결하게, 중요한 것만!

중요한 점은 CLAUDE.md 내용이 `/compact` 후에도 살아남는다는 것입니다. 세션 중 컨텍스트가 압축될 때 Claude가 디스크에서 파일을 다시 읽어 새롭게 주입합니다.

3. 파일 위치와 적용 범위 (4계층)

CLAUDE.md 파일은 네 곳에 위치할 수 있으며 광범위한 것부터 구체적인 순서로

나열됩니다. 관리 정책(조직 전체)은 모든 사용자에게 적용되고 어떤 설정으로도 제외할 수 없습니다. 사용자 수준은 개인 지시사항으로 내 모든 프로젝트에 적용됩니다. 프로젝트 수준은 저장소 루트에 위치하며 git으로 팀과 공유합니다. 서브디렉토리는 해당 디렉토리에만 범위가 한정되며 세션 시작이 아닌 Claude가 그 곳의 파일을 읽을 때 온디맨드로 로드됩니다.

우선순위 (높음 → 낮음):

1. C:\Program Files\ClaudeCode\CLAUDE.md ← 회사 정책 (IT 관리)
↓
2. ~/.claude/CLAUDE.md ← 내 개인 설정 (모든 프로젝트)
↓
3. ~/프로젝트/CLAUDE.md ← 이 프로젝트 (git 공유) ★ 핵심
↓
4. ~/프로젝트/backend/CLAUDE.md ← 서브디렉토리 (온디맨드)
 ~/프로젝트/frontend/CLAUDE.md
 ~/프로젝트/data/CLAUDE.md

더 구체적인 파일이 더 광범위한 파일보다 우선

버전 관리에 포함되면 안 되는 개인 선호도는 CLAUDE.local.md를 생성하고 .gitignore에 추가하세요. 에디터 특이사항, 선호하는 커밋 스타일, 임시 재정의는 공유 파일을 오염시키지 않고 여기에 넣습니다.

4. 무엇을 넣고 무엇을 빼야 하나

✅ 반드시 포함해야 할 것

CLAUDE.md는 세 가지 레이어로 나뉩니다: (1) 프로젝트가 무엇인지, (2) 왜 그 방식으로 작동하는지, (3) Claude가 어떻게 운영해야 하는지. 프로젝트 설명과 버전 번호가 포함된 기술 스택으로 시작하세요. Claude는 코드를 읽어서 많은 것을 추론할 수 있지만 Next.js 14 대신 15를 쓴다거나 Prisma 대신 Drizzle을 선택했다는 것은 추측하지 못합니다.

✅ 넣어야 할 것:

- 프로젝트 한 줄 설명 + 기술 스택 (버전 포함)
- 디렉토리 구조 맵 (전체 말고 핵심만)
- 빌드/테스트/실행 명령어 (코드 블록으로)
- Claude가 코드에서 추론 못하는 결정사항
- "절대 하지 말 것" 규칙
- 도메인 특화 지식 (업무 용어, 규칙)
- 자주 쓰는 워크플로우

❌ 넣으면 안 되는 것

CLAUDE.md가 너무 길면 Claude가 절반을 무시합니다. 중요한 규칙이 소음 속에 묻힙니다. 해결책: 가차없이 정리하세요. Claude가 지시 없이도 이미 올바르게 하는 것이 있다면 삭제하거나 Hook으로 전환하세요.

❌ 넣지 말 것:

- Claude가 이미 아는 표준 관행 ("PEP 8 따르세요")
- 코드에서 명확히 드러나는 것 ("React 사용 중")
- 너무 긴 배경 설명
- Hook으로 강제할 수 있는 것 (포매팅 등)
- 사용 안 하는 오래된 내용

5. 프로젝트별 최적 CLAUDE.md 파일

이제 스마트폰 불량·진단 분석 시스템에 맞는 실제 파일들을 만들어 드릴게요.

📄 파일 1: `~/.claude/CLAUDE.md` (개인 전역 설정)

markdown

개인 개발 환경 설정

내 역할 & 컨텍스트

- 제조업 스마트폰 품질/데이터 분석 담당
- Python 주력, 데이터 분석 + 웹 대시보드 개발
- Windows 11 + WSL2 (Ubuntu) + VS Code 환경

응답 방식 선호도

- 한국어로 답변
- 코드 먼저, 설명은 필요할 때만
- 대안이 있으면 반드시 언급
- 불확실한 내용은 "확인 필요"로 표시

공통 개발 스타일

- Python: 타입 힌트 필수, docstring 필수
- 커밋: Conventional Commits (feat/fix/data/chore)
- 에러 처리: 구체적인 예외 타입 사용
- 로깅: print() 대신 logging 모듈

환경

- Python: 3.12 (pyenv)
- 패키지 관리: uv
- 컨테이너: Docker + Docker Compose
- DB: PostgreSQL 16, Redis 7

 파일 2: ~/smartphone-analytics/CLAUDE.md (프로젝트 공유)

markdown

Smartphone Analytics System

프로젝트 개요

스마트폰 불량·진단 데이터를 수집·분석하고
웹 대시보드로 실시간 제공하는 시스템.
품질팀과 경영진이 불량 패턴을 파악해
생산 의사결정을 지원하는 것이 목표.

기술 스택 (버전 고정)

- **Backend**: FastAPI 0.115 + Python 3.12
- **Database**: PostgreSQL 16 + Redis 7 (캐싱)
- **Frontend**: React 18 + TypeScript 5.4
- **차트**: Plotly.js 2.x + Recharts 2.x
- **데이터**: Pandas 2.2 + Scikit-learn 1.5 + Scipy 1.13
- **ORM**: SQLAlchemy 2.0 + Alembic (마이그레이션)
- **인증**: JWT (access 15분 + refresh 7일)
- **배포**: Docker Compose (개발) / K8s (운영 예정)

디렉토리 구조

```
smartphone-analytics/
├── backend/
│   ├── api/      # FastAPI 라우터 (엔드포인트별 파일)
│   ├── models/   # SQLAlchemy ORM 모델
│   ├── schemas/  # Pydantic 요청/응답 스키마
│   ├── services/ # 비즈니스 로직 레이어
│   ├── analysis/ # 데이터 분석 모듈 ★ 핵심
│   │   ├── defect.py # 불량 패턴 탐지
│   │   ├── battery.py # 배터리 분석
│   │   ├── anomaly.py # 이상치 탐지
│   │   └── forecast.py # 불량률 예측
│   ├── db/       # DB 연결, 세션 관리
│   ├── core/     # 설정, 보안, 공통 유틸
│   └── tests/    # pytest 테스트
├── frontend/
│   ├── src/
│   │   ├── components/ # 공통 UI 컴포넌트
│   │   ├── pages/      # 페이지 컴포넌트
│   │   ├── charts/     # Plotly/Recharts 래퍼
│   │   ├── hooks/      # 커스텀 React Hooks
│   │   ├── api/        # API 호출 함수
│   │   └── types/      # TypeScript 타입 정의
├── data/
│   ├── raw/         # 원본 진단 데이터 (git 제외)
│   ├── processed/   # 전처리 완료 데이터
│   └── samples/     # 테스트용 샘플 데이터
├── migrations/     # Alembic 마이그레이션
├── docker-compose.yml
└── .env.example
```

```
## 핵심 명령어
```bash
개발 환경 시작
docker compose up -d # DB + Redis 실행
cd backend && uvicorn main:app --reload --port 8000
cd frontend && npm run dev # 포트 3000

전체 실행
docker compose --profile full up -d

테스트
pytest backend/tests/ -v --cov=backend
pytest backend/tests/analysis/ -v # 분석 모듈만

DB 마이그레이션
alembic revision --autogenerate -m "설명"
alembic upgrade head
alembic downgrade -1 # 롤백

패키지 추가
uv pip install 패키지명
uv pip freeze > requirements.txt

린트/포맷
ruff check backend/ --fix
ruff format backend/
...

도메인 지식 (필독)
```

### ### 불량 유형 분류

HW\_BATTERY 배터리 관련 (충전불량, 급방전, 팽창)  
HW\_DISPLAY 화면 관련 (불량화소, 터치불량, 번인)  
HW\_CAMERA 카메라 관련 (초점불량, 화질저하)  
HW\_SENSOR 센서 오작동 (자이로, 가속도, GPS)  
SW\_CRASH 앱/OS 크래시  
SW\_PERF 성능 저하 (발열, 렉)  
INTERMITTENT 간헐적 오류 (재현 어려움)

### ### 진단 데이터 핵심 컬럼

device\_id 기기 고유 ID (UUID)  
model\_code 모델 코드 (예: SM-A556, SM-S928)  
manufacture\_date 제조일 (YYYY-MM-DD)  
battery\_cycle 배터리 충방전 횟수  
battery\_health 배터리 건강도 (%)  
cpu\_temp\_max 최고 CPU 온도 (°C)  
ram\_usage\_avg 평균 RAM 사용률 (%)  
defect\_type 불량 유형 코드 (위 분류표)  
severity 심각도 1-5 (5=최고)  
diagnosed\_at 진단 일시

### ### 핵심 비즈니스 지표

불량률(DPM) =  $\text{불량 기기수} / \text{총 생산수} \times 1,000,000$   
MTBF = 평균 무고장 시간 (일 단위)  
진단 정확도 =  $\text{올바른 진단} / \text{전체 진단} \times 100$   
모델별 불량 분포 = 각 모델의 불량 유형별 비율  
조기 불량 = 30일 이내 발생 불량 (특별 모니터링)



```

코딩 컨벤션

Python (백엔드/분석)
```python
# ✅ 올바른 패턴
async def get_defect_rate(
    model_code: str,
    start_date: date,
    end_date: date,
    db: AsyncSession = Depends(get_db)
) -> DefectRateResponse:
    """
    특정 모델의 기간별 불량률을 계산합니다.

    Args:
        model_code: 스마트폰 모델 코드 (예: SM-A556)
        start_date: 분석 시작일
        end_date: 분석 종료일

    Returns:
        DefectRateResponse: 불량률, 불량수, 총 기기수

    Raises:
        HTTPException 404: 모델을 찾을 수 없을 때
    """
    ...

# ✅ 분석 함수 패턴
def detect_battery_anomaly(
    df: pd.DataFrame,
    threshold: float = 1.5
) -> pd.DataFrame:
    """IQR 방식으로 배터리 이상치 탐지"""
    ...

```

```

### TypeScript (프론트엔드)
```typescript
// ✅ API 응답 타입 항상 정의
interface DefectRateResponse {
 model_code: string;
 defect_rate_dpm: number;
 total_devices: number;
 defect_count: number;
 period: { start: string; end: string };
}

// ✅ 차트 컴포넌트 Props 타입화
interface DefectTrendChartProps {
 data: DefectRateResponse[];
 height?: number;
 onPointClick?: (date: string) => void;
}
...

API 설계 원칙

```

GET /api/v1/defects/rate      불량률 조회  
 GET /api/v1/defects/trend      트렌드 조회  
 GET /api/v1/devices/{id}/diagnosis 기기 진단 이력  
 POST /api/v1/diagnosis/upload      진단 데이터 업로드  
 GET /api/v1/analysis/anomaly      이상치 탐지 결과  
 GET /api/v1/dashboard/summary      대시보드 요약 KPI

- 모든 응답: {"data": ..., "meta": {...}, "error": null}
- 페이지네이션: ?page=1&size=20
- 날짜 필터: ?start\_date=2026-01-01&end\_date=2026-05-01
- 모델 필터: ?model\_code=SM-A556

### ## 보안 & 데이터 정책

- 고객 개인정보(이름, 연락처)는 DB에 저장하지 않음
- **device\_id**는 **UUID**로만 식별, 실제 시리얼 노출 금지
- 분석 결과 외부 노출 시 모델명 마스킹 (내부용만 전체 공개)
- **API 인증: JWT Bearer** 토큰 필수 (공개 엔드포인트 제외)

### ## 절대 하지 말 것

- **raw/** 폴더 데이터 **git** 커밋 금지
- **.env** 파일 커밋 금지 (**.env.example**만 허용)
- **main** 브랜치 직접 푸시 금지 (**PR** 필수)
- 분석 결과 검증 없이 대시보드 배포 금지
- 하드코딩된 **DB** 접속 정보 금지
- **print()** 디버깅 코드 커밋 금지

### ## 현재 개발 상태 (2026-05)

- ✅ 완료: **DB** 모델, 기본 **CRUD API**
- 🔄 진행: 분석 모듈 (배터리 이상치 탐지)
- ⏳ 예정: **React** 대시보드, 실시간 알림

📄 파일 3: backend/CLAUDE.md (백엔드 서브디렉토리)

markdown

---

paths:

- backend/\*\*

---

## # 백엔드 세부 규칙

### ## FastAPI 패턴

- 모든 엔드포인트: `async def` 사용
- DB 세션: `Depends(get_async_db)` 의존성 주입
- 응답 모델: 반드시 `response_model` 지정
- 예러: `HTTPException` 사용, 상세 메시지 포함

### ## 분석 모듈 규칙 (analysis/)

- 순수 함수 우선 (사이드 이펙트 없음)
- 입력: `DataFrame`, 출력: `DataFrame` or `dict`
- 대용량 데이터: `chunking` 처리 (10만 행 단위)
- 모든 분석 함수: 실행 시간 로깅 필수

### ## 테스트 규칙

- 분석 함수: 샘플 데이터로 단위 테스트 필수
- API: `TestClient`로 통합 테스트
- fixtures: `tests/conftest.py`에 공통 정의
- 커버리지 목표: 85% 이상

 파일 4: frontend/CLAUDE.md (프론트엔드 서브디렉토리)

markdown

---

paths:

- frontend/\*\*

---

## # 프론트엔드 세부 규칙

### ## 상태 관리

- 서버 데이터: TanStack Query (useQuery/useMutation)
- 로컬 UI 상태: useState/useReducer
- 전역 상태: Zustand (필요 최소한으로)

### ## 차트 규칙 (charts/)

- 시계열 트렌드: Recharts LineChart
- 비율/분포: Plotly Pie/Bar
- 지도/히트맵: Plotly Heatmap
- 모든 차트: 로딩 스켈레톤 + 에러 상태 처리 필수

### ## 스타일

- Tailwind CSS 유틸리티 클래스 우선
- 커스텀 CSS는 최소화
- 반응형: mobile-first (sm/md/lg 브레이크포인트)
- 다크모드: dark: 클래스 모든 컴포넌트에 적용

📄 파일 5: CLAUDE.local.md (개인용, gitignore)

markdown

## # 로컬 개발 환경 (git 제외)

### ## 내 로컬 설정

- DB 비밀번호: (개인 로컬 DB용)
- 테스트 서버: <http://192.168.1.xxx:8000>
- 개인 API 키 위치: ~/.secrets/

### ## 현재 내가 작업 중인 것

- 배터리 수명 예측 모델 (battery.py)
- 막힌 부분: LSTM vs ARIMA 선택

### ## 임시 메모

- 2026-05-10: 데이터팀 요청 - 모델별 조기불량을 추가
- 다음 PR: anomaly.py 완성 후 PR #42

## 6. 효과적인 CLAUDE.md 작성 원칙

### 원칙 1: 구체성 (Specificity)

Claude는 코드를 읽어서 많은 것을 추론할 수 있지만 Next.js 15 대신 14를 쓴다거나 Prisma 대신 Drizzle을 선택했다는 것은 추측하지 못합니다. 디렉토리 구조 맵을 포함하세요. 모든 파일이 아닌 간단한 설명이 있는 최상위 레이아웃만 있으면 됩니다.

markdown

## # ❌ 너무 모호

"Python 베스트 프랙티스를 따르세요"

## # ✅ 구체적

"분석 함수는 반드시 `pd.DataFrame`을 입력받고  
{`'result': df, 'stats': dict, 'warnings': list`}  
형태의 `dict`를 반환해야 합니다"

### 원칙 2: 간결성 (Brevity)

CLAUDE.md가 너무 길면 Claude가 절반을 무시합니다. Claude가 지시 없이도 이미 올바르게 하는 것이 있다면 삭제하거나 Hook으로 전환하세요.

길이 기준:  
전역 **CLAUDE.md**: 50줄 이하  
프로젝트 **CLAUDE.md**: 150줄 이하 ★  
서브디렉토리 **CLAUDE.md**: 30줄 이하

원칙 3: 살아있는 문서 (Living Document)

```
bash

작업 후 Claude에게 직접 업데이트 요청
"오늘 결정한 내용들 (배터리 분석에 LSTM 사용,
이상치 임계값 1.5 IQR)을 CLAUDE.md에 추가해줘"

또는 /memory 명령어로 자동 관리
```

7. CLAUDE.md vs 다른 설정의 역할 분담

설정	역할	예시
CLAUDE.md	"무엇을, 왜, 어떻게" 규칙	기술스택, 컨벤션, 도메인 지식
Hooks	자동 강제 실행	저장 시 린트, 위험 명령 차단
Memory	Claude가 자동 학습	반복 수정 패턴, 예러 해결법
Skills	특화 워크플로우	불량 보고서 생성, DB 쿼리 작성
MCP	외부 도구 연결	PostgreSQL 직접 조회, GitHub

8. 첫 시작: `/init` 후 정제 흐름

```
bash

cd ~/smartphone-analytics

1단계: Claude가 프로젝트 분석해서 초안 생성
claude /init

2단계: 생성된 CLAUDE.md 열어서 확인
code CLAUDE.md

3단계: 위의 템플릿 내용으로 보강
(도메인 지식, 불량 유형, 핵심 지표 등 추가)

4단계: 서브디렉토리 파일 생성
touch backend/CLAUDE.md frontend/CLAUDE.md

5단계: 개인 로컬 파일 생성 + gitignore 등록
touch CLAUDE.local.md
echo "CLAUDE.local.md" >> .gitignore
```

잘 설정된 CLAUDE.md 하나로 매 세션마다 "우리 프로젝트는요..."를 반복하는 대신 바로 "배터리 이상치 탐지 함수 만들어줘"로 시작할 수 있게 됩니다. 막히는 부분이 있으면 알려주세요! 🚀

---

## Claude Code Checkpoint 시스템 완벽 가이드

### 1. Checkpoint란?

Claude Code에는 두 가지 안전 메커니즘이 있습니다. Checkpoint는 파일 변경을 되돌릴 수 있게 하고, Permissions는 Claude가 물어보지 않고 할 수 있는 것을 제어합니다. 모든 파일 편집은 되돌릴 수 있습니다. Claude가 파일을 편집하기 전에 현재 내용을 스냅샷합니다. 문제가 생기면 Esc를 두 번 눌러 이전 상태로 되돌리거나 Claude에게 undo를 요청할 수 있습니다. Checkpoint는 세션에 로컬이며 git과는 별개입니다.

가장 좋은 비유:

`/rewind`의 핵심 가치는 실수할 용기를 주는 것입니다.



git과의 차이:

**git:**

장기 버전 관리  
의도적으로 커밋  
전체 저장소 추적  
팀 공유 가능  
커밋 메시지 작성 필요

**Checkpoint:**

세션 내 즉각적 실행 취소  
매 **Claude** 응답마다 자동 생성  
**Claude** 편집 도구만 추적  
내 세션에만 로컬  
완전 자동

→ 둘은 대체제가 아닌 보완 관계

"중요 마일스톤마다 **git commit**,  
그 사이 세밀한 실험은 **Checkpoint**"

## 2. Checkpoint의 내부 동작 원리

### 자동 스냅샷 시스템

파일 스냅샷 시스템이라는 우아한 메커니즘이 그 뒤에 있습니다. 각 Claude 응답 후 Claude Code는 Claude가 수정한 모든 파일을 자동으로 스냅샷합니다. 이 과정은 완전히 투명하게 이루어지며 사용자는 이것이 일어나는 것을 알아차리지도 못합니다. 파일 백업은 증분 방식입니다. 실제로 변경된 파일만 새 버전을 얻습니다. 두 턴 사이에 파일이 변경되지 않았다면 이전 백업이 재사용되어 저장 공간을 절약합니다.

Claude가 파일 편집하기 직전:

대화 흐름:

---

턴 1: "배터리 분석 함수 만들어줘"

- 📷 Checkpoint A 자동 생성
- battery.py 생성

---

턴 2: "IQR 대신 Z-Score 방식으로 바꿔줘"

- 📷 Checkpoint B 자동 생성
- battery.py 수정

---

턴 3: "관련 테스트도 업데이트해줘"

- 📷 Checkpoint C 자동 생성
  - test\_battery.py 수정
- 

/rewind → A, B, C 중 선택해서 복원

## 저장 위치

각 메시지, 도구 사용, 결과는 ~/.claude/projects/ 아래의 플레인텍스트 JSONL 파일에 기록됩니다. 이것이 세션 되감기, 재개, 분기를 가능하게 합니다. Claude가 코드를 변경하기 전에 영향받는 파일도 스냅샷하여 필요하면 되돌릴 수 있습니다.

```
~/.claude/projects/
└─ smartphone-analytics/
 └─ sessions/
 └─ 2026-05-10-abc123/
 ├── conversation.jsonl ← 전체 대화 기록
 ├── checkpoints/
 │ ├── checkpoint_a3f8d2/ ← 턴 1 스냅샷
 │ │ └─ battery.py
 │ ├── checkpoint_b7c4e1/ ← 턴 2 스냅샷
 │ │ └─ battery.py
 │ └─ checkpoint_c9a2f6/ ← 턴 3 스냅샷
 │ └─ test_battery.py
 └─ metadata.json
```

### 3. Checkpoint 활성화 방법 3가지

#### 방법 1: `Esc` × 2 (가장 빠름)

```
Claude가 작업 중이거나 완료 직후:
Esc → Esc

→ 즉시 Checkpoint 선택 UI 표시
```

#### 방법 2: `/rewind` 명령어 (별칭: `/checkpoint`)

```
bash

/rewind # Checkpoint 목록 표시
/rewind # 별칭: /checkpoint (동일)
```

`/rewind`(또한 `/checkpoint`, 또는 `Esc` 두 번)는 대화를 이전 지점으로 롤백할 수 있게 합니다.

#### 방법 3: Claude에게 직접 요청

```
"방금 작업 되돌려줘"
"이전 상태로 복원해줘"
"마지막 변경 undo해줘"
```

### 4. `/rewind` 실행 시 선택 UI

`/rewind`를 실행하면 세 가지 복원 옵션을 제공합니다:

/rewind 실행 후 표시되는 UI:

어느 시점으로 되돌릴까요?

☐

checkpoint\_c9a2f6

10:42 AM

"테스트 업데이트"

☐

checkpoint\_b7c4e1

10:35 AM

"Z-Score 방식 적용"

☒

checkpoint\_a3f8d2

10:28 AM

"배터리 분석 함수"

← 선택

복원 방식:

1. 대화 + 파일 모두 복원 ← 가장 많이 사용

2. 대화만 복원 (파일 유지)

3. 파일만 복원 (대화 유지)

### 3가지 복원 모드 상세

모드	파일 상태	대화 상태	사용 시점
대화 + 파일 모두	✅ 복원	✅ 복원	완전히 다른 방향으로 재시작
대화만 복원	❌ 그대로	✅ 복원	코드는 좋지만 대화 맥락 정리 필요
파일만 복원	✅ 복원	❌ 그대로	코드만 되돌리고 대화는 계속

### 5. Checkpoint의 핵심 한계 (반드시 알아야 함)

결정적인 한계가 있습니다. Bash 명령어는 추적되지 않습니다. Claude가 rm, mv, cp를 실행하면 그 변경은 영구적입니다. Checkpoint는 Claude의 파일 편집 도구를 통한 편집만 캡처합니다.

✓ Checkpoint가 추적하는 것:

파일 편집 (**Write, Edit, MultiEdit** 도구)  
새 파일 생성  
편집 도구를 통한 파일 삭제

✗ Checkpoint가 추적하지 않는 것:

**Bash** 명령어 (**rm, mv, cp, git** 등)  
사용자가 직접 수동으로 편집한 내용  
다른 동시 세션의 변경  
**DB/API/외부** 시스템 변경  
배포된 코드

Checkpoint는 세션에 로컬이며 git과는 별개입니다. 파일 변경만 커버합니다. 데이터베이스, API, 배포처럼 원격 시스템에 영향을 미치는 액션은 checkpoint할 수 없으며, 이것이 Claude가 외부 부수 효과가 있는 명령어를 실행하기 전에 물어보는 이유입니다.

python

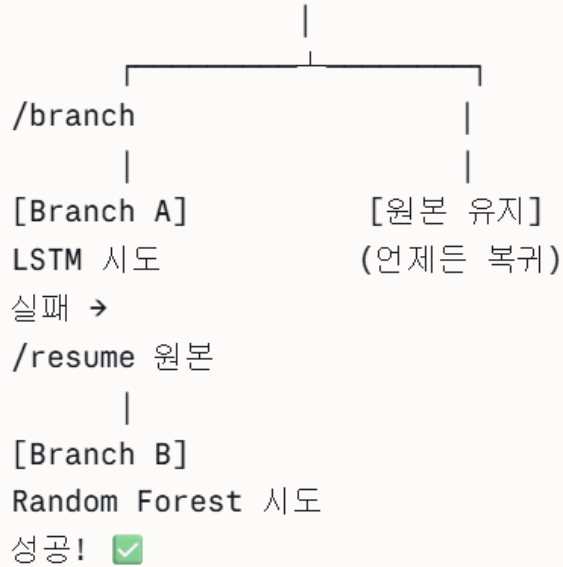
```
✓ 복원 가능 (Edit 도구 사용)
Claude가 battery.py를 직접 편집

✗ 복원 불가능 (Bash 명령어)
Claude가 실행: rm -rf data/processed/
Claude가 실행: alembic upgrade head (DB 변경)
Claude가 실행: git push origin main
```

## 6. `/branch` (구 `/fork`): 타임라인 분기

`/branch` 명령어(이전 `/fork`, 별칭 유지)는 현재 타임라인을 교체하면서 원래 기록에 접근 가능하게 유지합니다. 새 타임라인은 별도의 JSONL 로그로 독립적으로 분기됩니다.

현재 상태: 불량 탐지 모델 v1 완성 (작동 중)



## /branch 실전 사용법

```
bash

1단계: 현재 좋은 상태에서 분기
/branch battery-lstm-experiment

2단계: 마음껏 실험
"LSTM 기반 배터리 수명 예측 모델 구현해줘"
"하이퍼파라미터 튜닝해줘"
→ 실패하거나 마음에 안 들면

3단계: 원본으로 돌아가기
/resume # 분기 전 상태 선택

4단계: 다른 방향으로 재시도
/branch battery-rf-experiment
"Random Forest로 배터리 수명 예측해줘"
```

## 7. Checkpoint vs Branch vs Rewind 관계도

시간 흐름 →

[A]——[B]——[C]——[D] (현재)

↑

자동 Checkpoint

↑

자동 Checkpoint

/rewind → B 선택:

[A]——[B] (여기서 재시작, C/D는 삭제)

/branch → C에서 분기:

[A]——[B]——[C]——[D] (원본)

└─[E]—[F] (실험 Branch)

/rewind + "대화만":

파일: [D] 상태 유지

대화: [B] 시점으로 복원

## 8. API에서 Checkpoint 사용 (개발자용)

파일 복원은 대화 자체를 되감지 않습니다. `rewindFiles()`(TypeScript) 또는 `rewind_files()`(Python) 호출 후에도 대화 기록과 컨텍스트는 그대로 유지됩니다.

```
python

Python SDK로 Checkpoint 제어
import anthropic

async def safe_refactor_with_checkpoint():
 async with ClaudeSDKClient(
 ClaudeAgentOptions(
 enable_file_checkpointing=True, # Checkpoint 활성화
 resume=session_id # 이전 세션 재개
)
) as client:
 # 작업 시작
 await client.query("배터리 분석 모듈 리팩토링해줘")

 checkpoint_id = None
 async for message in client.receive_response():
 # Checkpoint ID 수집
 if message.type == "checkpoint":
 checkpoint_id = message.checkpoint_id

 # 결과가 마음에 안 들면 파일만 되돌리기
 if not satisfied:
 await client.rewind_files(checkpoint_id)
 # 대화는 유지, 파일만 복원
```

```
typescript

// TypeScript SDK
const client = new ClaudeSDKClient({
 enableFileCheckpointing: true
});

// 파일 복원 (대화는 유지)
await client.rewindFiles(checkpointId);

// 세션 전체 포크
const forkedSession = await client.forkSession(checkpointId);
```

## 9. Checkpoint 비활성화



```
bash

환경 변수로 비활성화
export CLAUDE_CODE_DISABLE_FILE_CHECKPOINTING=1

또는 전역 설정에서
~/.claude/settings.json
{
 "fileCheckpointingEnabled": false
}
```

비활성화 시 `/rewind`는 여전히 대화를 롤백할 수 있지만 파일은 복원할 수 없습니다.

비활성화는 보통 권장하지 않습니다. 저장 공간이 매우 부족한 경우가 아니면 꺼두는 것이 좋습니다.

## 10. 실전 패턴: 프로젝트에 적용

스마트폰 불량 분석 프로젝트 기준 실전 시나리오입니다.

## 패턴 1: 대규모 리팩토링 전 안전망

시나리오: 불량 탐지 API 전체 구조 변경

### Before:

"분석 모듈 전체를 비동기로 리팩토링해줘"

→ 망함, 이전으로 돌아가고 싶음

→ Esc Esc → 전체 복원

### After (안전한 방식):

1. `git commit` (중요 시점 저장)

2. "계획만 먼저 보여줘" (`/plan`)

3. 계획 승인 후 실행

4. 중간에 이상하면 → Esc Esc

5. 성공하면 → `git commit`

## 패턴 2: A/B 실험 (Branch 활용)

"배터리 이상치 탐지에 IQR vs Z-Score 중 뭐가 더 좋을지 실험해줘"

Step 1: `/branch iqr-approach`

→ IQR 구현 + 테스트 실행 → 정확도 87%

Step 2: `/resume` (원본 복귀)

`/branch zscore-approach`

→ Z-Score 구현 + 테스트 실행 → 정확도 91%

Step 3: Z-Score가 더 좋음 → 해당 Branch 채택

### 패턴 3: 파일만 복원, 대화는 유지

시나리오: Claude가 코드를 잘못 수정했지만  
대화에서 쌓인 컨텍스트(30분치)는 유지하고 싶음

/rewind → "파일만 복원" 선택

결과:

- battery.py: 이전 상태로 복원 
- 대화 컨텍스트: 그대로 유지 
- "이번엔 다른 방식으로 구현해줘" 계속 진행

### 패턴 4: Hook과 연동 (자동 Checkpoint 전략)

```
json
// .claude/settings.json
{
 "hooks": {
 "PreToolUse": [{
 "matcher": "Write|Edit|MultiEdit",
 "hooks": [{
 "type": "command",
 "command": "bash -c 'echo \"⚠️ 파일 수정 전 Checkpoint 자동 생성 중...\",'",
 "async": true
 }]
 }]
 }
}
```

## 11. Checkpoint + git 최적 워크플로우

Checkpoint는 코드 리뷰, 테스트, 버전 관리의 대체가 아닌 보완입니다. 모든 회귀에 대한 보호를 보장하지 않으며 수동 편집이나 Claude 외 명령어는 커버하지 않습니다.

최적 워크플로우:

---

① 기능 시작

```
git checkout -b feature/battery-anomaly
```

---

② 바이브 코딩 (Checkpoint 자동 생성)

"배터리 이상치 탐지 함수 만들어줘"

[Checkpoint A 자동]

"테스트 추가해줘"

[Checkpoint B 자동]

→ 실패 → Esc Esc → Checkpoint A로 복원

"다른 방식으로 다시 해줘"

[Checkpoint C 자동]

→ 성공! 

---

③ 중요 마일스톤마다 git commit

```
git add -A
```

```
git commit -m "feat(analysis): add battery anomaly detection"
```

---

④ 더 복잡한 변경 전

```
/branch battery-lstm-experiment
```

→ 실험 후 성공 시 main branch에 merge

---

⑤ 기능 완성

```
git push origin feature/battery-anomaly
```

→ PR 생성

---

Checkpoint = 세션 내 안전망 (초 단위)

git = 프로젝트 히스토리 (장기)

## 12. 전체 명령어 요약

명령어	단축키	동작	사용 시점
<code>/rewind</code>	<code>Esc×2</code>	과거 Checkpoint로 복원	잘못된 방향 되돌리기
<code>/checkpoint</code>	—	<code>/rewind</code> 의 별칭	동일
<code>/branch [이름]</code>	—	현재 시점에서 분기 생성	위험한 실험 전
<code>/fork [이름]</code>	—	<code>/branch</code> 의 구 별칭	동일 (하위 호환)
<code>/resume</code>	—	세션/분기 선택해서 재개	이전 세션·분기 이어서

다음으로 Claude Code의 Plan Mode 심화, Permission 시스템 (도구 권한 관리), 실전 바이브 코딩 세션 시연, 또는 배포 자동화 (GitHub Actions + Claude) 중 어떤 것을 알아보을까요? 🚀

# RAG (검색 증강 생성) 심화 완벽 가이드

## 1. RAG란? — 근본 원리

RAG는 LLM을 외부 지식으로 grounding하는 지배적인 아키텍처입니다. 모델을 코퍼스에 파인튜닝하는 대신(비용이 많이 들고, 느리고, 업데이트하기 어려운) RAG는 쿼리 시점에 관련 정보를 검색해 LLM에 컨텍스트로 제공합니다.

LLM만 사용 (문제):

질문: "2026년 5월 SM-S928 불량률은?"

LLM: "알 수 없습니다" 또는 "환각(Hallucination)" ❌

RAG (해결):

질문 → 내부 DB 검색 → 관련 데이터 추출

→ LLM에 주입 → 정확한 답변 ✅

## 핵심 파이프라인 (5단계)

- |            |                      |
|------------|----------------------|
| ① Index    | 문서를 벡터로 변환해 저장       |
| ② Query    | 사용자 질문을 벡터로 변환       |
| ③ Retrieve | 의미적으로 유사한 문서 검색      |
| ④ Augment  | 검색 결과를 프롬프트에 주입      |
| ⑤ Generate | LLM이 컨텍스트 기반으로 답변 생성 |

## 2. Naive RAG의 실패 — 왜 기초 RAG는 부족한가

2024년에는 PDF 몇 개를 벡터 DB에 밀어 넣고 코사인 유사도 검색을 실행하면 충분했습니다. 2026년에는 그 접근법, 즉 Naive RAG가 최선의 경우 프로토타입, 최악의 경우 부채로 여겨집니다. 병목은 더 이상 "생성" 부분이 아닙니다. 실패 지점은 거의 항상 검색(Retrieval)입니다.

표준 RAG 튜토리얼은 보여줍니다: 문서 임베딩 → 벡터 DB 저장 → 상위 k개 검색 → 생성. 데모에서는 작동합니다. 프로덕션에서는 세 가지 이유로 무너집니다: 의미적 갭(사용자 쿼리와 문서가 다른 어휘 사용), 컨텍스트 창 오염(10개 청크 검색 시 2개만 관련 있어 신호 희석), 그리고 관계적 갭(멀티홉 추론이 필요한 질문)입니다.

Naive RAG 실패율: 약 40% (2026 기준)

실패 유형:

① 의미적 갭 (Semantic Gap)

질문: "구독 취소 방법"

문서 제목: "계정 해지 정책"

→ 다른 단어라 못 찾음 ✖

② 컨텍스트 오염 (Context Pollution)

10개 청크 검색 → 2개만 관련

→ 나머지 8개가 LLM을 혼란시킴 ✖

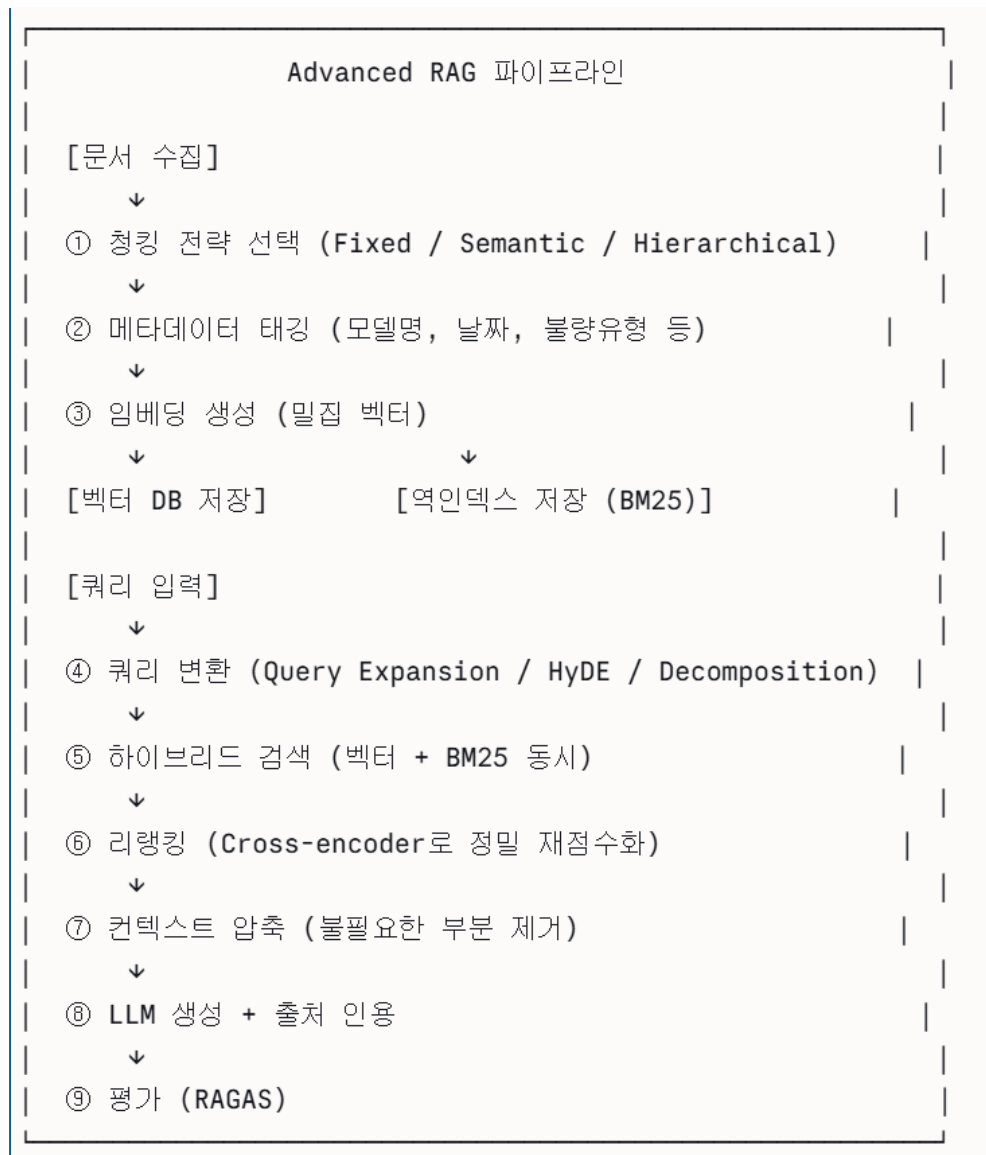
③ 멀티홉 실패 (Multi-hop Failure)

"SM-S928의 배터리 부량이 생산 라인과  
연관된 이유는?"

→ 단일 청크에 답이 없음 ✖

### 3. Advanced RAG 전체 아키텍처

2026년 기준 하이브리드 검색과 크로스 인코더 리랭킹이 기본 스택이 되었으며, 단일 방법 검색 대비 관련성과 사실성에서 일관되게 성능을 앞섭니다.



## 4. 핵심 기술 ①: 청킹 전략 (Chunking)

RAG 실패의 80%는 LLM이 아닌 수집과 청킹 레이어로 거슬러 올라갑니다. 대부분의 팀은 프롬프트 튜닝과 모델 교체에 수 주를 쓰면서 검색이 조용히 세 번에 한 번씩 틀린 컨텍스트를 반환하는 동안 이것을 발견합니다.

### 4-1. Fixed-Size Chunking (고정 크기)

재귀적 문자 분할 또는 512토큰 고정 크기 청킹을 사용하세요. 코퍼스가 의미적 경계에서 이득을 본다는 구체적 증거가 없다면 이것이 기본값입니다.



```
python

from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
 chunk_size=512, # 토큰 수
 chunk_overlap=50, # 청크 간 겹침 (문맥 연속성)
 separators=["\n\n", "\n", ".", " "]
)
chunks = splitter.split_text(document)
```

## 4-2. Semantic Chunking (의미 기반)

고정 문자 수로 분할하는 대신 의미적 청킹은 임베딩 유사도를 사용해 주제 경계를 감지합니다. 목표는 의미적으로 완전한 청크를 만드는 것입니다. 각 청크는 스스로 질문에 답할 수 있어야 합니다.

```
python

from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

chunker = SemanticChunker(
 OpenAIEmbeddings(),
 breakpoint_threshold_type="percentile", # 상위 95% 유사도 경계에서 분할
 breakpoint_threshold_amount=95
)
chunks = chunker.split_text(document)
```

## 4-3. Hierarchical Chunking (계층적) ★ 프로젝트 추천

진단 보고서 계층 구조:

Level 0 (문서 전체 요약): 128 토큰

↓

Level 1 (섹션 요약): 256 토큰

- | └─ 배터리 분석 섹션
- | └─ 화면 불량 섹션
- | └─ 센서 오작동 섹션

↓

Level 2 (세부 청크): 512 토큰

- | └─ 구체적 측정값
- | └─ 진단 로그

→ 큰 질문 → Level 0/1 검색

→ 세부 질문 → Level 2 검색

python

```
def hierarchical_chunk(document: str) -> dict:
 """계층적 청킹 구현"""
 return {
 "summary": summarize(document, max_tokens=128),
 "sections": [
 {
 "title": section.title,
 "summary": summarize(section.text, max_tokens=256),
 "chunks": fixed_chunk(section.text, size=512)
 }
 for section in extract_sections(document)
]
 }
```

## 5. 핵심 기술 ②: 하이브리드 검색 (Hybrid Search)

하이브리드 검색은 밀집 벡터와 키워드 매칭(BM25)을 결합합니다. 기업 배포에서 밀집 검색만 사용하는 것보다 20~40% 높은 검색 재현율을 보였습니다. 정확한 용어, 약어, 도메인 특화 전문어에 특히 효과적이어서 기업 지식베이스에서 사실상 필수가 되었습니다.

벡터 검색 (Dense):

"배터리 이상"

→ 의미적으로 유사한 것

→ 다른 표현도 찾을 수 있음

BM25 키워드 검색 (Sparse):

"battery\_health < 80"

→ 정확한 용어 매칭

→ 오타/동의를 못 찾을 수 있음

하이브리드 = 둘 다 검색 후 점수 결합

→ 정확한 용어 + 의미 유사성 모두 커버

python

```
from weaviate.classes.query import HybridFusion
```

```
def hybrid_search(query: str, alpha: float = 0.5) -> list:
```

```
 """
```

```
 alpha: 0.0 = 순수 BM25, 1.0 = 순수 벡터
```

```
 0.5 = 균형 (기본값)
```

```
 도메인 전문 용어 많을수록 alpha 낮춤
```

```
 """
```

```
 results = collection.query.hybrid(
 query=query,
 alpha=alpha,
 fusion_type=HybridFusion.RELATIVE_SCORE,
 limit=20 # 리랭킹 전 충분한 후보 확보
)
```

```
 return results
```

```
프로젝트 특화: 불량 유형 코드는 정확 매칭 중요
```

```
→ alpha=0.3 (BM25 70%, 벡터 30%)
```

```
defect_results = hybrid_search(
 "HW_BATTERY SM-S928 2026-04",
 alpha=0.3
)
```

## 6. 핵심 기술 ③: 쿼리 변환 (Query Transformation)

Naive RAG는 단순히 쿼리 벡터를 문서 벡터와 매칭합니다. 쿼리 변환은 검색 전에 쿼리를 개선합니다.

### 6-1. HyDE (Hypothetical Document Embeddings)

```
python

def hyde_search(user_query: str) -> list:
 """
 실제 답변처럼 생긴 가상 문서를 먼저 생성 →
 그 문서로 검색 (실제 DB 문서와 더 유사한 벡터 생성)
 """
 # 1단계: 가상 답변 생성
 hypothetical_doc = llm.generate(
 f"다음 질문에 대한 이상적인 답변을 작성하세요:\n{user_query}"
)

 # 2단계: 가상 답변으로 검색
 results = vector_db.search(hypothetical_doc)
 return results

예시:
질문: "SM-S928 배터리 불량 원인은?"
가상 문서: "SM-S928의 배터리 불량은 주로 충방전 사이클
500회 이후 발생하며 배터리 건강도 80% 미만..."
→ 이 가상 문서로 실제 DB 검색 → 더 관련성 높은 결과
```

### 6-2. Query Decomposition (쿼리 분해)

python

```
def decompose_and_search(complex_query: str) -> dict:
 """
 복잡한 질문을 단순한 하위 질문으로 분해
 """
 # 예: "SM-S928과 SM-A556의 배터리 불량률을
 # 2025년과 2026년을 비교해줘"

 sub_queries = llm.generate(f"""
 다음 복잡한 질문을 독립적인 하위 질문들로 분해하세요:
 {complex_query}
 JSON 배열로 반환하세요.
 """)

 # 분해 결과:
 # ["SM-S928 2025년 배터리 불량률?",
 # "SM-S928 2026년 배터리 불량률?",
 # "SM-A556 2025년 배터리 불량률?",
 # "SM-A556 2026년 배터리 불량률?"]

 # 각 하위 질문 병렬 검색
 results = {}
 for q in sub_queries:
 results[q] = hybrid_search(q)

 return results
```

## 7. 핵심 기술 ④: 리랭킹 (Reranking) ★ 가장 임팩트 큼

리랭킹을 절대적으로 필수로 만드세요. 크로스 인코더 리랭킹의 18~42% 정밀도 향상은 전체 RAG 파이프라인에서 가장 레버리지가 높은 최적화입니다.

리랭킹은 초기 검색 후 적용되는 두 번째 관련성 점수화 단계입니다. 크로스 인코더 모델은 사용자 쿼리와 각 검색된 청크를 단일 입력으로 읽어 각 쌍에 대한 정밀한 관련성 점수를 생성합니다. 상위 3~5개의 가장 높은 점수를 받은 청크가 LLM에 전달됩니다. 리랭킹은 일반적으로 20개 후보에 대해 150~300ms의 추가 지연이 발생합니다.

검색 단계 흐름:

[하이브리드 검색] → 상위 20개 후보

↓

[Bi-encoder: 속도] → 빠른 1차 필터링

↓

[Cross-encoder: 정밀도] → 쿼리+청크 쌍 정밀 점수화

↓

[상위 3~5개] → LLM 컨텍스트에 주입

python

```
from sentence_transformers import CrossEncoder

크로스 인코더 (정밀 리랭킹)
reranker = CrossEncoder(
 "cross-encoder/ms-marco-MiniLM-L-6-v2"
)

def rerank(query: str, candidates: list[str]) -> list[str]:
 """
 초기 검색 결과를 크로스 인코더로 정밀 재점수화
 """
 # 쿼리-청크 쌍 점수화
 pairs = [[query, chunk] for chunk in candidates]
 scores = reranker.predict(pairs)

 # 점수 기준 재정렬
 ranked = sorted(
 zip(candidates, scores),
 key=lambda x: x[1],
 reverse=True
)

 # 상위 5개만 반환
 return [chunk for chunk, score in ranked[:5]]
```

```
★ LLM Lost-in-the-Middle 문제 해결
def order_for_llm(chunks: list) -> list:
 """
 LLM은 앞과 뒤에 더 집중함 (가운데 정보 손실)
 → 가장 관련성 높은 것을 앞과 뒤에 배치
 """
 if len(chunks) <= 2:
 return chunks
 most_relevant = chunks[0]
 rest = chunks[1:]
 # 두 번째로 중요한 것을 마지막에 배치
 return [most_relevant] + rest[:-1] + [rest[-1]]
```

## 8. 핵심 기술 ⑤: GraphRAG (지식 그래프 RAG)

GraphRAG는 복잡한 추론 작업에서 벡터 전용 접근법을 앞질렀으며, 50% 낮은 비용으로 우월한 정확도를 달성했습니다.

지식 그래프 RAG(GraphRAG라고도 함)는 벡터 검색을 엔티티와 관계를 나타내는 구조화된 지식 그래프로 보강합니다. 사용자가 두 개 이상의 관련 사실을 연결해야 하는 멀티홉 질문을 할 때 순수 벡터 검색은 종종 실패합니다. 단일 청크에 완전한 답이 없기 때문입니다.

벡터 RAG의 한계:

질문: "SM-S928의 배터리 불량률 특정 생산 라인과  
연관된 이유가 뭔가요?"

벡터 검색:

청크 A: "SM-S928 배터리 불량률 5.2%"

청크 B: "3번 라인 납땜 공정 이슈"

→ 두 청크가 연결되지 않음 ❌

GraphRAG:

SM-S928 → [생산품] → 3번 라인

→ [불량유형] → HW\_BATTERY

→ [원인] → 납땜 공정 불량

→ 연결 경로 탐색으로 멀티홉 질문 해결 ✅

python

# Neo4j 기반 GraphRAG 구현

from neo4j import GraphDatabase

class SmartphoneKnowledgeGraph:

def \_\_init\_\_(self, uri, user, password):  
 self.driver = GraphDatabase.driver(uri, auth=(user, password))

def build\_graph(self, defect\_data: list[dict]):  
 """진단 데이터로 지식 그래프 구축"""  
 with self.driver.session() as session:  
 for record in defect\_data:  
 session.run("""  
 MERGE (d:Device {model: \$model, serial: \$serial})  
 MERGE (l:ProductionLine {name: \$line})  
 MERGE (dt:DefectType {code: \$defect\_type})  
  
 MERGE (d)-[:PRODUCED\_ON]->(l)  
 MERGE (d)-[:HAS\_DEFECT]->(dt)  
  
 SET d.manufacture\_date = \$manufacture\_date  
 SET dt.severity = \$severity  
 """, \*\*record)

def multi\_hop\_query(self, start\_entity: str) -> dict:  
 """멀티홉 관계 탐색"""  
 with self.driver.session() as session:  
 result = session.run("""  
 MATCH (d:Device {model: \$model})  
 -[:HAS\_DEFECT]->(dt:DefectType)  
 -[:CAUSED\_BY]->(cause:RootCause)  
 -[:LINKED\_TO]->(l:ProductionLine)  
 RETURN d.model, dt.code, cause.description,  
 l.name, count(\*) as frequency  
 ORDER BY frequency DESC  
 """, model=start\_entity)  
 return result.data()





```

하이브리드 RAG + GraphRAG 결합
def combined_rag_search(query: str) -> str:
 # 1. 벡터 + BM25 하이브리드 검색
 vector_results = hybrid_search(query)

 # 2. 엔티티 추출
 entities = extract_entities(query)
 # ["SM-S928", "배터리 불량", "생산 라인"]

 # 3. 그래프 탐색으로 관계 추가
 graph_context = kg.multi_hop_query(entities[0])

 # 4. 두 결과 결합 후 리랭킹
 combined = vector_results + format_graph(graph_context)
 reranked = rerank(query, combined)

 return "\n".join(reranked)

```

## 9. 핵심 기술 ⑥: Agentic RAG

Agentic RAG: "리서치 에이전트"가 잠자는 동안 "검색 엔진"과 "벡터 DB"를 고용해 100페이지 보고서를 만들어주는 것입니다.

Agentic RAG는 LLM이 검색 계획을 생성하고, 중간 결과를 비판하고, 답변하기 전에 다시 검색할지 결정하는 Self-RAG를 포함합니다. 반복적 제어로 복잡한 QA를 개선합니다.

python

```
class AgenticRAG:
 """
 계획 → 검색 → 평가 → 재검색 루프
 """

 def answer(self, query: str, max_iterations: int = 3) -> str:
 context = []
 iteration = 0

 while iteration < max_iterations:
 # 1. 현재 상태로 답변 시도
 response = self.llm.generate(
 query=query,
 context=context,
 system="""답변할 수 있으면 답변하세요.
 더 많은 정보가 필요하면 검색 쿼리를 제안하세요.
 형식: {"action": "answer"|"search",
 "content": "답변 또는 검색어"}"""
)

 # 2. LLM 결정 파싱
 if response["action"] == "answer":
 return response["content"] # 답변 완성

 elif response["action"] == "search":
 # 3. 추가 검색 실행
 new_context = hybrid_search(response["content"])
 context.extend(new_context)
 iteration += 1

 # 최대 반복 후 최선의 답변 반환
 return self.llm.generate(query=query, context=context)
```

## 10. 메타데이터 필터링

메타데이터 필터링이 결합된 의미적 청킹은 특화된 컨텍스트에서 60% 정확도를 보여 의미적 청킹 단독보다 크게 앞섰습니다.

python

# 메타데이터를 청킹 시 함께 저장

```
def index_with_metadata(documents: list) -> None:
 for doc in documents:
 chunks = semantic_chunk(doc["content"])

 for chunk in chunks:
 vector_db.insert({
 "text": chunk,
 "embedding": embed(chunk),

 # 🔑 핵심 메타데이터
 "metadata": {
 "model_code": doc["model_code"], # SM-S928
 "defect_type": doc["defect_type"], # HW_BATTERY
 "severity": doc["severity"], # 1~5
 "manufacture_date": doc["manufacture_date"],
 "production_line": doc["line"],
 "doc_type": doc["type"] # diagnosis/report
 }
 })
```

```

필터와 함께 검색 (훨씬 정밀)
def filtered_search(
 query: str,
 model_code: str = None,
 defect_type: str = None,
 date_range: tuple = None
) -> list:

 filters = {}
 if model_code:
 filters["model_code"] = {"$eq": model_code}
 if defect_type:
 filters["defect_type"] = {"$eq": defect_type}
 if date_range:
 filters["manufacture_date"] = {
 "$gte": date_range[0],
 "$lte": date_range[1]
 }

 return vector_db.search(
 query=query,
 filter=filters,
 limit=20
)

사용 예시:
results = filtered_search(
 "배터리 수명 급감 원인",
 model_code="SM-S928",
 defect_type="HW_BATTERY",
 date_range=("2026-01-01", "2026-05-01")
)

```

## 11. 벡터 DB 선택 가이드

임베딩 모델은 검색이 의미를 얼마나 잘 포착하는지 결정합니다. 벡터 데이터베이스 선택은 지연, 비용, 운영 복잡성에 영향을 미칩니다.

DB	특징	추천 상황	가격
Pinecone	완전 관리형, 서버리스	인프라 관리 싫을 때	\$0.33/1M reads
Weaviate	하이브리드 검색 내장	키워드+벡터 필요	무료 ~
Qdrant	Rust 기반, 고성능	고처리량 워크로드	무료 ~
pgvector	PostgreSQL 확장	이미 PostgreSQL 사용 중	추가 비용 없음
Chroma	로컬 개발용	프로토타이핑	무료

**프로젝트 추천:** pgvector (이미 PostgreSQL 사용 + 추가 인프라 불필요)

```
python

PostgreSQL + pgvector 설정
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE diagnostic_embeddings (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 device_id UUID REFERENCES devices(id),
 chunk_text TEXT NOT NULL,
 embedding vector(1536), -- OpenAI text-embedding-3-small
 model_code VARCHAR(20),
 defect_type VARCHAR(20),
 severity INTEGER,
 doc_date DATE,
 created_at TIMESTAMPTZ DEFAULT NOW()
);

-- HNSW 인덱스 (고속 근사 검색)
CREATE INDEX ON diagnostic_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- 하이브리드 검색 쿼리
SELECT chunk_text,
 embedding <=> $1 AS vector_score, -- 벡터 유사도
 ts_rank(to_tsvector(chunk_text),
 plainto_tsquery($2)) AS bm25_score -- BM25 점수
FROM diagnostic_embeddings
WHERE model_code = 'SM-S928'
ORDER BY (0.5 * vector_score + 0.5 * bm25_score)
LIMIT 20;
```

## 12. RAG 평가: RAGAS 프레임워크

naive RAG 파이프라인은 검색에서 약 40% 실패합니다. LLM이 자신감 있고 잘 구조화된 답변을 생성하지만 잘못된 문서에 근거합니다. 이것을 측정해야 합니다.

```
python

from ragas import evaluate
from ragas.metrics import (
 faithfulness, # 답변이 검색된 컨텍스트에 충실한가?
 answer_relevancy, # 답변이 질문과 관련 있는가?
 context_precision, # 검색된 컨텍스트가 정밀한가?
 context_recall # 필요한 컨텍스트가 모두 검색되었나?
)

평가 데이터셋
test_cases = [
 {
 "question": "SM-S928의 2026년 배터리 불량률은?",
 "answer": rag_pipeline.answer("SM-S928의 2026년 배터리 불량률은?"),
 "contexts": retrieved_chunks,
 "ground_truth": "SM-S928의 2026년 배터리 불량률은 3.2%입니다."
 },
 # ... 더 많은 테스트 케이스
]

4개 지표 동시 평가
scores = evaluate(
 dataset=test_cases,
 metrics=[
 faithfulness, # 목표: > 0.85
 answer_relevancy, # 목표: > 0.80
 context_precision, # 목표: > 0.75
 context_recall # 목표: > 0.80
]
)

print(scores)
```



## 13. 프로젝트 최적화된 완전한 RAG 시스템

스마트폰 불량 분석 시스템에 바로 적용 가능한 구현입니다.

python

```
backend/rag/diagnostic_rag.py
```

```
import asyncio
```

```
from typing import Optional
```

```
import numpy as np
```

```
from anthropic import Anthropic
```

```
class SmartphoneDiagnosticRAG:
```

```
 """
```

```
 스마트폰 불량·진단 데이터 특화 RAG 시스템
```

```
 하이브리드 검색 + 리랭킹 + Claude 생성
```

```
 """
```

```
 def __init__(self, db_session, collection):
```

```
 self.db = db_session
```

```
 self.collection = collection
```

```
 self.client = Anthropic()
```

```
 self.reranker = CrossEncoder(
```

```
 "cross-encoder/ms-marco-MiniLM-L-6-v2"
```

```
)
```

```

async def index_diagnostic_data(
 self, records: list[dict]
) -> None:
 """진단 데이터를 청킹 + 임베딩 + 저장"""
 for record in records:
 # 1. 계층적 청킹
 chunks = self._create_chunks(record)

 # 2. 배치 임베딩 생성
 embeddings = self._embed_batch(
 [c["text"] for c in chunks]
)

 # 3. 메타데이터와 함께 저장
 for chunk, embedding in zip(chunks, embeddings):
 await self.db.execute("""
 INSERT INTO diagnostic_embeddings
 (chunk_text, embedding, model_code,
 defect_type, severity, doc_date)
 VALUES ($1, $2, $3, $4, $5, $6)
 """, chunk["text"], embedding,
 record["model_code"],
 record["defect_type"],
 record["severity"],
 record["date"])

```



```

async def answer(
 self,
 query: str,
 model_code: Optional[str] = None,
 defect_type: Optional[str] = None,
 date_range: Optional[tuple] = None
) -> dict:
 """
 질문에 대해 RAG 기반 정확한 답변 생성
 """
 # 1. 쿼리 변환 (HyDE)
 hypothetical = await self._hyde_expand(query)

 # 2. 하이브리드 검색 (벡터 + BM25 + 메타필터)
 candidates = await self._hybrid_search(
 hypothetical,
 model_code=model_code,
 defect_type=defect_type,
 date_range=date_range,
 limit=20
)

 # 3. 크로스 인코더 리랭킹
 reranked = self._rerank(query, candidates)[:5]

 # 4. LLM 위치 최적화
 ordered = self._order_for_llm(reranked)

 # 5. Claude로 최종 답변 생성
 context = "\n\n---\n\n".join(ordered)
 response = self.client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=1500,
 system="""당신은 스마트폰 품질 분석 전문가입니다.
주어진 진단 데이터를 기반으로 정확하게 답변하세요.
데이터에 없는 내용은 "데이터 없음"으로 표시하세요.
답변 마지막에 근거 데이터 출처를 명시하세요.""",
 messages=[{
 "role": "user",
 "content": f"""참고 데이터:
{context}

```



```

질문: {query}"""
 }]
)

 return {
 "answer": response.content[0].text,
 "sources": [c[:100] + "..." for c in ordered],
 "retrieved_count": len(candidates),
 "final_context_count": len(ordered)
 }

def _rerank(self, query: str, candidates: list) -> list:
 pairs = [[query, c] for c in candidates]
 scores = self.reranker.predict(pairs)
 ranked = sorted(
 zip(candidates, scores),
 key=lambda x: x[1], reverse=True
)
 return [c for c, _ in ranked]

def _order_for_llm(self, chunks: list) -> list:
 """Lost-in-Middle 방지: 가장 중요한 것을 앞뒤에"""
 if len(chunks) <= 2:
 return chunks
 return [chunks[0]] + chunks[2:] + [chunks[1]]

```

## 14. RAG 발전 단계 로드맵

현재 (2026):

Naive RAG → Advanced RAG → Agentic RAG → GraphRAG

단계별 개선 효과:

Naive RAG	기준 (실패율 40%)
+ 하이브리드 검색	+20~40% 재현율
+ 리랭킹	+18~42% 정밀도
+ 메타데이터 필터	+60% 특화 정확도
+ GraphRAG	멀티홉 질문 해결
+ Agentic RAG	자기 교정, 반복 검색

프로젝트 권장 도입 순서:

1주차: **pgvector** + 기본 하이브리드 검색 ← 지금 시작

2주차: 크로스 인코더 리랭킹 추가

3주차: 메타데이터 필터링 (모델/날짜/불량유형)

4주차: **RAGAS**로 성능 측정 → 최적화

2개월차: **GraphRAG** (생산라인-불량 연관 분석)

3개월차: **Agentic RAG** (자율 분석 리포트 생성)

다음으로 **Fine-tuning vs RAG 심층 비교**, **Claude API**를 활용한 **RAG 완전 구현 실습**, 또는 **프로젝트에서 실시간 불량 데이터 스트리밍 RAG 설계**로 이어갈까요?



## Claude API RAG 완전 구현 실습

### 스마트폰 불량·진단 데이터 분석 시스템

#### 전체 구조

```
smartphone-analytics/
├── backend/
│ ├── rag/
│ │ ├── __init__.py
│ │ ├── chunker.py # 청킹 전략
│ │ ├── embedder.py # 임베딩 생성
│ │ ├── retriever.py # 하이브리드 검색
│ │ ├── reranker.py # 리랭킹
│ │ ├── generator.py # Claude API 생성
│ │ └── pipeline.py # 전체 파이프라인
│ ├── api/
│ │ └── rag_router.py # FastAPI 엔드포인트
│ └── db/
│ └── schema.sql # pgvector 스키마
├── tests/
│ └── test_rag.py # 평가 테스트
├── scripts/
│ └── ingest_data.py # 데이터 수집 스크립트
└── .env
```

## STEP 1: 환경 설정

bash

# 패키지 설치

```
uv pip install \
anthropic \
psycpg2-binary pgvector \
sentence-transformers \
rank-bm25 \
langchain langchain-community \
python-dotenv \
fastapi uvicorn \
pandas numpy \
ragas datasets # 평가용
```

ini

# .env

ANTHROPIC\_API\_KEY=sk-ant-xxxxx

DATABASE\_URL=postgresql://analyst:password@localhost/analytics\_db

# 임베딩 모델 (로컬 실행, 무료)

EMBEDDING\_MODEL=sentence-transformers/all-MiniLM-L6-v2

EMBEDDING\_DIM=384

# RAG 설정

CHUNK\_SIZE=512

CHUNK\_OVERLAP=50

TOP\_K\_RETRIEVE=20

TOP\_K\_RERANK=5

HYBRID\_ALPHA=0.5 # 0=BM25만, 1=벡터만

## STEP 2: 데이터베이스 스키마

```
sql

-- db/schema.sql

-- pgvector 확장 활성화
CREATE EXTENSION IF NOT EXISTS vector;

-- 진단 문서 임베딩 테이블
CREATE TABLE diagnostic_embeddings (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

 -- 원본 텍스트
 chunk_text TEXT NOT NULL,
 chunk_index INTEGER NOT NULL, -- 문서 내 청크 순서

 -- 임베딩 벡터 (384차원, MiniLM 모델)
 embedding vector(384) NOT NULL,

 -- 메타데이터 (필터링용)
 model_code VARCHAR(20), -- SM-S928, SM-A556
 defect_type VARCHAR(20), -- HW_BATTERY, HW_DISPLAY
 severity INTEGER, -- 1~5
 doc_date DATE,
 doc_type VARCHAR(20), -- diagnosis, report, manual
 source_file TEXT, -- 원본 파일명

 -- BM25 전문 검색용
 ts_vector TSVECTOR -- 한국어 토큰화 벡터
 GENERATED ALWAYS AS (
 to_tsvector('simple', chunk_text)
) STORED,

 created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```

-- HNSW 인덱스 (벡터 근사 검색, 매우 빠름)
CREATE INDEX idx_embedding_hnsw
ON diagnostic_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- BM25 전문 검색 인덱스
CREATE INDEX idx_ts_vector
ON diagnostic_embeddings
USING gin(ts_vector);

-- 메타데이터 필터 인덱스
CREATE INDEX idx_model_code ON diagnostic_embeddings(model_code);
CREATE INDEX idx_defect_type ON diagnostic_embeddings(defect_type);
CREATE INDEX idx_doc_date ON diagnostic_embeddings(doc_date);

```

### STEP 3: 청킹 모듈

```

backend/rag/chunker.py

from dataclasses import dataclass
from typing import Optional
import re

@dataclass
class Chunk:
 text: str
 index: int
 metadata: dict

class DiagnosticChunker:
 """
 스마트폰 진단 데이터 특화 청킹 전략
 - 진단 보고서: 섹션 기반 분할
 - 로그 데이터: 고정 크기 분할
 """

```

- CSV 데이터: 행 기반 분할

"""

```
def __init__(
 self,
 chunk_size: int = 512,
 chunk_overlap: int = 50
):
 self.chunk_size = chunk_size
 self.chunk_overlap = chunk_overlap
```

```
def chunk_diagnostic_report(
 self,
 text: str,
 metadata: dict
) -> list[Chunk]:
 """
```

```
 진단 보고서 청킹
 섹션 헤더(##, **) 기준으로 우선 분할,
 섹션이 너무 크면 재귀 분할
 """
```

```
 # 1. 섹션 헤더로 1차 분할
 sections = self._split_by_section(text)
```

```
 chunks = []
```

```
 idx = 0
```

```
 for section in sections:
```

```
 if self._token_count(section) <= self.chunk_size:
```

```
 # 섹션이 청크 크기 이하면 그대로 사용
```

```
 chunks.append(Chunk(
 text=section.strip(),
 index=idx,
 metadata=metadata
))
```

```
 idx += 1
```

```
 else:
```



```

 # 크면 재귀 분할 + overlap 적용
 sub_chunks = self._recursive_split(section)
 for sub in sub_chunks:
 chunks.append(Chunk(
 text=sub.strip(),
 index=idx,
 metadata=metadata
))
 idx += 1

 return [c for c in chunks if len(c.text) > 50]

def chunk_csv_data(
 self,
 rows: list[dict],
 rows_per_chunk: int = 10,
 metadata: dict = None
) -> list[Chunk]:
 """
 CSV/테이블 데이터 청킹
 여러 행을 자연어 형태로 변환 후 묶음
 """
 chunks = []
 for i in range(0, len(rows), rows_per_chunk):
 batch = rows[i:i + rows_per_chunk]

 # 각 행을 자연어 문장으로 변환
 text_lines = []
 for row in batch:
 line = self._row_to_text(row)
 text_lines.append(line)

 chunk_text = "\n".join(text_lines)
 chunks.append(Chunk(
 text=chunk_text,
 index=i // rows_per_chunk,

```

```

 metadata=metadata or {}
))

 return chunks

def _row_to_text(self, row: dict) -> str:
 """
 DB 행을 자연어 문장으로 변환
 (벡터 검색 품질 향상)
 """
 parts = []

 if row.get("model_code"):
 parts.append(f"모델: {row['model_code']}")
 if row.get("defect_type"):
 parts.append(f"불량유형: {row['defect_type']}")
 if row.get("severity"):
 parts.append(f"심각도: {row['severity']}")
 if row.get("battery_health"):
 parts.append(f"배터리건강도: {row['battery_health']}%")
 if row.get("cpu_temp_max"):
 parts.append(f"최고온도: {row['cpu_temp_max']}°C")
 if row.get("diagnosed_at"):
 parts.append(f"진단일: {row['diagnosed_at']}")

 return " | ".join(parts)

def _split_by_section(self, text: str) -> list[str]:
 """마크다운 헤더 기준 섹션 분할"""
 pattern = r'(?=#{1,3}Ws|(?<=Wn)W*W*[^*]+W*W*:)'
 sections = re.split(pattern, text)
 return [s for s in sections if s.strip()]

def _recursive_split(self, text: str) -> list[str]:
 """재귀적 문자 분할 (LangChain 방식)"""
 separators = ["\n\n", "\n", ". ", " "]

```

```

 return self._split_recursive(text, separators)

 def _split_recursive(
 self,
 text: str,
 separators: list[str]
) -> list[str]:
 if self._token_count(text) <= self.chunk_size:
 return [text]

 separator = separators[0] if separators else " "
 splits = text.split(separator)

 chunks = []
 current = ""
 for split in splits:
 if self._token_count(current + separator + split) >
 <= self.chunk_size:
 current += separator + split
 else:
 if current:
 chunks.append(current.strip())
 current = split

 if current:
 chunks.append(current.strip())

 # Overlap 적용
 return self._apply_overlap(chunks)

 def _apply_overlap(self, chunks: list[str]) -> list[str]:
 if self.chunk_overlap == 0 or len(chunks) < 2:
 return chunks

 result = [chunks[0]]
 for i in range(1, len(chunks)):

```

```

 # 이전 청크 마지막 부분을 현재 청크 앞에 붙임
 prev_end = chunks[i-1][-self.chunk_overlap:]
 result.append(prev_end + " " + chunks[i])

 return result

 @staticmethod
 def _token_count(text: str) -> int:
 """간단한 토큰 수 추정 (단어 수 × 1.3)"""
 return int(len(text.split()) * 1.3)

```

#### STEP 4: 임베딩 모듈

```

backend/rag/embedder.py

import os
import numpy as np
from sentence_transformers import SentenceTransformer
from typing import Union
import asyncio
from functools import lru_cache

class DiagnosticEmbedder:
 """
 로컬 임베딩 모델 (무료, 빠름)
 sentence-transformers/all-MiniLM-L6-v2
 384차원, 속도/품질 균형 최적
 """

 def __init__(
 self,
 model_name: str = "all-MiniLM-L6-v2"
):
 print(f"임베딩 모델 로딩: {model_name}")
 self.model = SentenceTransformer(model_name)

```

```

self.dimension = self.model.get_sentence_embedding_dimension()
print(f"모델 준비 완료. 차원: {self.dimension}")

def embed_single(self, text: str) -> list[float]:
 """단일 텍스트 임베딩"""
 embedding = self.model.encode(
 text,
 normalize_embeddings=True # 코사인 유사도용 정규화
)
 return embedding.tolist()

def embed_batch(
 self,
 texts: list[str],
 batch_size: int = 64,
 show_progress: bool = True
) -> list[list[float]]:
 """
 배치 임베딩 (대량 데이터 처리)
 배치 단위로 처리해 메모리 효율화
 """
 all_embeddings = []

 for i in range(0, len(texts), batch_size):
 batch = texts[i:i + batch_size]
 embeddings = self.model.encode(
 batch,
 normalize_embeddings=True,
 show_progress_bar=show_progress and len(texts) > 100
)
 all_embeddings.extend(embeddings.tolist())

 if show_progress:
 progress = min(i + batch_size, len(texts))
 print(f"임베딩 진행: {progress}/{len(texts)}")

```

```

 return all_embeddings

 def embed_query(self, query: str) -> list[float]:
 """
 쿼리 임베딩 (문서와 동일한 방식)
 쿼리 앞에 접두어 추가로 검색 품질 향상
 """
 prefixed = f"검색 질문: {query}"
 return self.embed_single(prefixed)

```

## STEP 5: 하이브리드 검색 모듈

```

backend/rag/retriever.py

import os
from typing import Optional
import psycopg2
from psycopg2.extras import RealDictCursor
import json
from .embedder import DiagnosticEmbedder

class HybridRetriever:
 """
 하이브리드 검색: 벡터 유사도 + BM25 키워드
 pgvector 기반, 추가 인프라 없음
 """

 def __init__(self, db_url: str, embedder: DiagnosticEmbedder):
 self.conn = psycopg2.connect(db_url)
 self.embedder = embedder

 def retrieve(
 self,
 query: str,
 top_k: int = 20,

```

```

alpha: float = 0.5, # 0=BM25만, 1=벡터만
메타데이터 필터
model_code: Optional[str] = None,
defect_type: Optional[str] = None,
date_from: Optional[str] = None,
date_to: Optional[str] = None,
doc_type: Optional[str] = None
) -> list[dict]:
 """
 하이브리드 검색 실행
 벡터 검색 + BM25 점수를 RRF로 결합
 """

 # 1. 쿼리 임베딩
 query_embedding = self.embedder.embed_query(query)

 # 2. 메타데이터 필터 조건 생성
 filters, params = self._build_filters(
 model_code, defect_type, date_from, date_to, doc_type
)

 # 3. 하이브리드 검색 쿼리
 # RRF(Reciprocal Rank Fusion)로 두 점수 결합
 sql = f"""
 WITH vector_search AS (
 -- 벡터 유사도 검색
 SELECT
 id,
 chunk_text,
 model_code,
 defect_type,
 severity,
 doc_date,
 doc_type,
 source_file,
 1 - (embedding <=> %s::vector) AS vector_score,
 ROW_NUMBER() OVER (

```

```

 ORDER BY embedding <=> %s::vector
) AS vector_rank
FROM diagnostic_embeddings
WHERE 1=1 {filters}
ORDER BY embedding <=> %s::vector
LIMIT %s
),
bm25_search AS (
 -- BM25 키워드 검색
 SELECT
 id,
 chunk_text,
 model_code,
 defect_type,
 severity,
 doc_date,
 doc_type,
 source_file,
 ts_rank_cd(ts_vector,
 plainto_tsquery('simple', %s)) AS bm25_score,
 ROW_NUMBER() OVER (
 ORDER BY ts_rank_cd(ts_vector,
 plainto_tsquery('simple', %s)) DESC
) AS bm25_rank
 FROM diagnostic_embeddings
 WHERE ts_vector @@ plainto_tsquery('simple', %s)
 {filters}
 ORDER BY bm25_score DESC
 LIMIT %s
),
combined AS (
 -- RRF 결합: 1/(k + rank) 방식
 SELECT
 COALESCE(v.id, b.id) AS id,
 COALESCE(v.chunk_text, b.chunk_text) AS chunk_text,
 COALESCE(v.model_code, b.model_code) AS model_code,

```



```

 COALESCE(v.defect_type, b.defect_type) AS defect_type,
 COALESCE(v.severity, b.severity) AS severity,
 COALESCE(v.doc_date, b.doc_date) AS doc_date,
 COALESCE(v.doc_type, b.doc_type) AS doc_type,
 COALESCE(v.source_file, b.source_file) AS source_file,
 COALESCE(v.vector_score, 0) AS vector_score,
 COALESCE(b.bm25_score, 0) AS bm25_score,
 -- RRF 점수 계산
 (
 {alpha} * COALESCE(1.0/(60 + v.vector_rank), 0) +
 (1-{alpha}) * COALESCE(1.0/(60 + b.bm25_rank), 0)
) AS rrf_score
 FROM vector_search v
 FULL OUTER JOIN bm25_search b ON v.id = b.id
)
SELECT * FROM combined
ORDER BY rrf_score DESC
LIMIT %s
"""

```

# 파라미터 구성

```
embedding_str = "[" + ",".join(map(str, query_embedding)) + "]"
```

```

all_params = (
 [embedding_str, embedding_str] # vector_search용
 + params # vector_search 필터
 + [top_k * 2] # vector_search LIMIT
 + [query, query, query] # bm25_search용
 + params # bm25_search 필터
 + [top_k * 2] # bm25_search LIMIT
 + [top_k] # 최종 LIMIT
)

```

```

with self.conn.cursor(cursor_factory=RealDictCursor) as cur:
 cur.execute(sql, all_params)
 results = cur.fetchall()

```

```

 return [dict(r) for r in results]

def _build_filters(
 self,
 model_code, defect_type, date_from, date_to, doc_type
) -> tuple[str, list]:
 """동적 필터 조건 생성"""
 conditions = []
 params = []

 if model_code:
 conditions.append("AND model_code = %s")
 params.append(model_code)
 if defect_type:
 conditions.append("AND defect_type = %s")
 params.append(defect_type)
 if date_from:
 conditions.append("AND doc_date >= %s")
 params.append(date_from)
 if date_to:
 conditions.append("AND doc_date <= %s")
 params.append(date_to)
 if doc_type:
 conditions.append("AND doc_type = %s")
 params.append(doc_type)

 return " ".join(conditions), params

```

## STEP 6: 리랭킹 모듈

```

backend/rag/reranker.py

from sentence_transformers import CrossEncoder
import numpy as np

```

```

class DiagnosticReranker:
 """
 크로스 인코더 리랭킹
 18~42% 정밀도 향상 (가장 임팩트 큰 최적화)
 """

 def __init__(
 self,
 model_name: str = "cross-encoder/ms-marco-MiniLM-L-6-v2"
):
 print(f"리랭킹 모델 로딩: {model_name}")
 self.model = CrossEncoder(model_name)

 def rerank(
 self,
 query: str,
 candidates: list[dict],
 top_k: int = 5
) -> list[dict]:
 """
 후보 청크들을 쿼리와의 정밀 관련성으로 재정렬
 """

 if not candidates:
 return []

 if len(candidates) == 1:
 return candidates

 # 쿼리-청크 쌍 점수화
 pairs = [
 [query, c["chunk_text"]]
 for c in candidates
]
 scores = self.model.predict(pairs)

 # 점수와 함께 정렬

```

```

 ranked = sorted(
 zip(candidates, scores),
 key=lambda x: x[1],
 reverse=True
)

 # 상위 k개 반환 (점수 포함)
 result = []
 for chunk, score in ranked[:top_k]:
 chunk_with_score = dict(chunk)
 chunk_with_score["rerank_score"] = float(score)
 result.append(chunk_with_score)

 return result

def order_for_llm(self, chunks: list[dict]) -> list[dict]:
 """
 LLM Lost-in-the-Middle 문제 완화
 가장 관련성 높은 청크를 앞과 뒤에 배치
 LLM은 컨텍스트의 앞과 뒤에 더 집중하는 경향
 """
 if len(chunks) <= 2:
 return chunks

 # 1번: 앞에 (가장 관련성 높음)
 # 마지막: 뒤에 (두 번째로 관련성 높음)
 # 나머지: 가운데
 return [chunks[0]] + chunks[2:] + [chunks[1]]

```

## STEP 7: Claude API 생성 모듈 (핵심)

```

backend/rag/generator.py

import anthropic
from typing import Optional, Generator
import json

```

```
class DiagnosticAnswerGenerator:
```

```
 """
```

```
 Claude API를 사용한 답변 생성
 스마트폰 불량 분석 도메인 특화
 """
```

```
 # 시스템 프롬프트 (도메인 특화)
```

```
 SYSTEM_PROMPT = """당신은 스마트폰 품질 관리 전문가입니다.
진단 데이터와 불량 분석 보고서를 기반으로 정확하고 실용적인 답변을
제공합니다.
```

```
 답변 원칙:
```

1. 제공된 컨텍스트에 있는 정보만 사용하세요
2. 데이터에 없는 내용은 "제공된 데이터에 없습니다"라고 명시하세요
3. 수치는 정확하게 인용하고 출처를 표시하세요
4. 불량 패턴이나 원인 분석은 근거를 명확히 하세요
5. 답변 끝에 참조한 데이터 출처를 요약하세요

```
 불량 유형 코드:
```

- HW\_BATTERY: 배터리 관련 (충전불량, 급방전)
- HW\_DISPLAY: 화면 관련 (불량화소, 터치불량)
- HW\_CAMERA: 카메라 관련
- HW\_SENSOR: 센서 오작동
- SW\_CRASH: 앱/OS 크래시
- SW\_PERF: 성능 저하
- INTERMITTENT: 간헐적 오류"""

```
 def __init__(self, api_key: Optional[str] = None):
 self.client = anthropic.Anthropic(api_key=api_key)
```

```
 def generate(
 self,
 query: str,
 context_chunks: list[dict],
 model: str = "claude-sonnet-4-6",
```

```

 max_tokens: int = 1500,
 temperature: float = 0.1 # 사실적 답변용 낮은 temperature
) -> dict:
 """
 검색된 컨텍스트를 기반으로 답변 생성
 """

 # 컨텍스트 포매팅
 context = self._format_context(context_chunks)

 # 프롬프트 구성
 user_message = f"""다음 진단 데이터와 보고서를 참고하여 질문에
 답변하세요.

 <context>
 {context}
 </context>

 <question>
 {query}
 </question>

 답변: """

 # Claude API 호출
 response = self.client.messages.create(
 model=model,
 max_tokens=max_tokens,
 temperature=temperature,
 system=self.SYSTEM_PROMPT,
 messages=[{
 "role": "user",
 "content": user_message
 }]
)

 answer = response.content[0].text

```

```

 return {
 "answer": answer,
 "model": model,
 "input_tokens": response.usage.input_tokens,
 "output_tokens": response.usage.output_tokens,
 "sources": self._extract_sources(context_chunks)
 }

def generate_stream(
 self,
 query: str,
 context_chunks: list[dict],
 model: str = "claude-sonnet-4-6"
) -> Generator[str, None, None]:
 """
 스트리밍 방식 답변 생성 (실시간 대시보드용)
 """

 context = self._format_context(context_chunks)

 user_message = f"""<context>
{context}
</context>

<question>{query}</question>

답변: """

 # 스트리밍 API 호출
 with self.client.messages.stream(
 model=model,
 max_tokens=1500,
 system=self.SYSTEM_PROMPT,
 messages=[{"role": "user", "content": user_message}]
) as stream:
 for text in stream.text_stream:

```

```
yield text
```

```
def generate_structured(
 self,
 query: str,
 context_chunks: list[dict]
) -> dict:
 """
 구조화된 JSON 형태로 답변 생성
 대시보드 차트 데이터 생성에 활용
 """

 context = self._format_context(context_chunks)

 user_message = f"""<context>
{context}
</context>

<question>{query}</question>

다음 JSON 형식으로만 답변하세요 (다른 텍스트 없이):
{{
 "summary": "핵심 답변 요약 (2~3문장)",
 "key_findings": ["주요 발견사항 1", "발견사항 2"],
 "metrics": {{
 "지표명": "값 (단위)"
 }},
 "recommendations": ["권고사항 1", "권고사항 2"],
 "data_coverage": "답변의 데이터 커버리지 설명",
 "confidence": "high|medium|low"
}}"""

 response = self.client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=1000,
 temperature=0.0, # 구조화 출력엔 0
 system="당신은 스마트폰 품질 분석 전문가입니다. 요청된 JSON
```



형식만 반환하세요.",

```
 messages=[{"role": "user", "content": user_message}]
)
```

```
raw = response.content[0].text.strip()
```

```
JSON 펜스 제거
```

```
raw = raw.replace("```json", "").replace("```", "").strip()
```

```
try:
```

```
 return json.loads(raw)
```

```
except json.JSONDecodeError:
```

```
 return {
```

```
 "summary": raw,
```

```
 "key_findings": [],
```

```
 "metrics": {},
```

```
 "recommendations": [],
```

```
 "confidence": "low"
```

```
 }
```

```
def _format_context(self, chunks: list[dict]) -> str:
```

```
 """청크를 LLM 프롬프트용으로 포맷팅"""
```

```
 parts = []
```

```
 for i, chunk in enumerate(chunks, 1):
```

```
 # 메타데이터 헤더
```

```
 header_parts = []
```

```
 if chunk.get("model_code"):
```

```
 header_parts.append(f"모델: {chunk['model_code']}")
```

```
 if chunk.get("defect_type"):
```

```
 header_parts.append(f"불량유형: {chunk['defect_type']}")
```

```
 if chunk.get("doc_date"):
```

```
 header_parts.append(f"날짜: {chunk['doc_date']}")
```

```
 if chunk.get("doc_type"):
```

```
 header_parts.append(f"문서유형: {chunk['doc_type']}")
```

```
 header = " | ".join(header_parts) if header_parts else "일반 데이터"
```

```

 parts.append(
 f"[문서 {i}] {header}\n{chunk['chunk_text']}"
)

 return "\n\n---\n\n".join(parts)

def _extract_sources(self, chunks: list[dict]) -> list[dict]:
 """답변에 사용된 출처 정보 추출"""
 sources = []
 for chunk in chunks:
 source = {
 "model_code": chunk.get("model_code"),
 "defect_type": chunk.get("defect_type"),
 "doc_date": str(chunk.get("doc_date", "")),
 "doc_type": chunk.get("doc_type"),
 "relevance_score": chunk.get("rerank_score", 0)
 }
 sources.append(source)
 return sources

```

## STEP 8: 전체 파이프라인 통합

```

backend/rag/pipeline.py

import os
from typing import Optional, Generator
from .chunker import DiagnosticChunker, Chunk
from .embedder import DiagnosticEmbedder
from .retriever import HybridRetriever
from .reranker import DiagnosticReranker
from .generator import DiagnosticAnswerGenerator
import psycpg2
from psycpg2.extras import execute_values
import time

```

```
class SmartphoneRAGPipeline:
```

```
 """
```

```
 스마트폰 불량 분석 RAG 완전 파이프라인
```

```
 인덱싱 + 검색 + 생성 통합
```

```
 """
```

```
 def __init__(self, db_url: str, anthropic_api_key: str):
```

```
 print("RAG 파이프라인 초기화 중...")
```

```
 self.embedder = DiagnosticEmbedder()
```

```
 self.chunker = DiagnosticChunker(
```

```
 chunk_size=512,
```

```
 chunk_overlap=50
```

```
)
```

```
 self.retriever = HybridRetriever(db_url, self.embedder)
```

```
 self.reranker = DiagnosticReranker()
```

```
 self.generator = DiagnosticAnswerGenerator(anthropic_api_key)
```

```
 self.conn = psycopg2.connect(db_url)
```

```
 print("RAG 파이프라인 준비 완료 ✅")
```

```
 #
```

---

```
 # 인덱싱 (데이터 수집)
```

```
 #
```

---

```
 def index_report(
```

```
 self,
```

```
 text: str,
```

```
 metadata: dict
```

```
) -> int:
```

```
 """
```

```
 진단 보고서 텍스트를 청킹 + 임베딩 + DB 저장
```

반환: 저장된 청크 수

"""

# 1. 청킹

chunks = self.chunker.chunk\_diagnostic\_report(text, metadata)

if not chunks:

return 0

# 2. 배치 임베딩

texts = [c.text for c in chunks]

embeddings = self.embedder.embed\_batch(texts)

# 3. DB 저장 (배치 인서트)

records = []

for chunk, embedding in zip(chunks, embeddings):

```
 records.append((
 chunk.text,
 chunk.index,
 "[" + ",".join(map(str, embedding)) + "]",
 metadata.get("model_code"),
 metadata.get("defect_type"),
 metadata.get("severity"),
 metadata.get("doc_date"),
 metadata.get("doc_type", "report"),
 metadata.get("source_file", "unknown")
))
```

with self.conn.cursor() as cur:

```
 execute_values(cur, """
 INSERT INTO diagnostic_embeddings
 (chunk_text, chunk_index, embedding,
 model_code, defect_type, severity,
 doc_date, doc_type, source_file)
 VALUES %s
 """, records)
```

self.conn.commit()

```

 return len(records)

 def index_csv_data(
 self,
 rows: list[dict],
 metadata: dict
) -> int:
 """CSV/DB 데이터를 청킹 + 임베딩 + 저장"""
 chunks = self.chunker.chunk_csv_data(rows, metadata=metadata)
 return self._save_chunks(chunks, metadata)

 def _save_chunks(
 self,
 chunks: list[Chunk],
 metadata: dict
) -> int:
 texts = [c.text for c in chunks]
 embeddings = self.embedder.embed_batch(texts)

 records = [
 (
 c.text, c.index,
 "[" + ",".join(map(str, emb)) + "]",
 metadata.get("model_code"),
 metadata.get("defect_type"),
 metadata.get("severity"),
 metadata.get("doc_date"),
 metadata.get("doc_type", "data"),
 metadata.get("source_file", "db")
)
 for c, emb in zip(chunks, embeddings)
]

 with self.conn.cursor() as cur:
 execute_values(cur, """
 INSERT INTO diagnostic_embeddings

```

```
 (chunk_text, chunk_index, embedding,
 model_code, defect_type, severity,
 doc_date, doc_type, source_file)
 VALUES %s
 """ , records)
 self.conn.commit()
 return len(records)
```

```
#
```

---

```
질의응답 (핵심)
```

```
#
```

---

```
def ask(
 self,
 query: str,
 model_code: Optional[str] = None,
 defect_type: Optional[str] = None,
 date_from: Optional[str] = None,
 date_to: Optional[str] = None,
 top_k: int = 5,
 alpha: float = 0.5,
 structured: bool = False
) -> dict:
 """
 질문 → 하이브리드 검색 → 리랭킹 → Claude 생성
 """
 start_time = time.time()

 # 1. 하이브리드 검색 (상위 20개 후보)
 candidates = self.retriever.retrieve(
 query=query,
 top_k=20,
```

```

 alpha=alpha,
 model_code=model_code,
 defect_type=defect_type,
 date_from=date_from,
 date_to=date_to
)

 retrieve_time = time.time() - start_time

 if not candidates:
 return {
 "answer": "관련 데이터를 찾을 수 없습니다. ",
 "다른 검색어나 필터를 시도해보세요.",
 "sources": [],
 "retrieved_count": 0,
 "latency_ms": int((time.time() - start_time) * 1000)
 }

 # 2. 크로스 인코더 리랭킹 (상위 5개로 압축)
 reranked = self.reranker.rerank(query, candidates, top_k=top_k)

 # 3. LLM 위치 최적화 (Lost-in-Middle 방지)
 ordered = self.reranker.order_for_llm(reranked)

 rerank_time = time.time() - start_time - retrieve_time

 # 4. Claude API로 답변 생성
 if structured:
 result = self.generator.generate_structured(query, ordered)
 result["type"] = "structured"
 else:
 result = self.generator.generate(query, ordered)
 result["type"] = "text"

 generate_time = time.time() - start_time - retrieve_time - rerank_time

```

```

성능 지표 추가
result.update({
 "retrieved_count": len(candidates),
 "final_context_count": len(ordered),
 "latency_ms": int((time.time() - start_time) * 1000),
 "breakdown_ms": {
 "retrieve": int(retrieve_time * 1000),
 "rerank": int(rerank_time * 1000),
 "generate": int(generate_time * 1000)
 }
})

return result

def ask_stream(
 self,
 query: str,
 **filters
) -> Generator[str, None, None]:
 """스트리밍 방식 답변 (실시간 대시보드용)"""
 candidates = self.retriever.retrieve(
 query=query, top_k=20, **filters
)
 if not candidates:
 yield "관련 데이터를 찾을 수 없습니다."
 return

 reranked = self.reranker.rerank(query, candidates, top_k=5)
 ordered = self.reranker.order_for_llm(reranked)

 yield from self.generator.generate_stream(query, ordered)

```

## STEP 9: FastAPI 엔드포인트

```
backend/api/rag_router.py
```



```

from fastapi import APIRouter, Query, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from typing import Optional
import os
from ..rag.pipeline import SmartphoneRAGPipeline

router = APIRouter(prefix="/api/v1/rag", tags=["RAG"])

파이프라인 싱글톤
_pipeline = None

def get_pipeline() -> SmartphoneRAGPipeline:
 global _pipeline
 if _pipeline is None:
 _pipeline = SmartphoneRAGPipeline(
 db_url=os.getenv("DATABASE_URL"),
 anthropic_api_key=os.getenv("ANTHROPIC_API_KEY")
)
 return _pipeline

class QuestionRequest(BaseModel):
 query: str
 model_code: Optional[str] = None
 defect_type: Optional[str] = None
 date_from: Optional[str] = None
 date_to: Optional[str] = None
 top_k: int = 5
 alpha: float = 0.5
 structured: bool = False

@router.post("/ask")
async def ask_question(req: QuestionRequest):
 """

```

RAG 기반 질문 답변

예시 질문:

- "SM-S928의 2026년 배터리 불량률 추이를 분석해줘"
  - "HW\_BATTERY 불량률의 주요 원인은?"
  - "이번 달 심각도 4~5인 불량 기기 목록"
- """

```
pipeline = get_pipeline()
result = pipeline.ask(
 query=req.query,
 model_code=req.model_code,
 defect_type=req.defect_type,
 date_from=req.date_from,
 date_to=req.date_to,
 top_k=req.top_k,
 alpha=req.alpha,
 structured=req.structured
)
return result
```

@router.get("/ask/stream")

```
async def ask_stream(
 query: str = Query(..., description="질문"),
 model_code: Optional[str] = None,
 defect_type: Optional[str] = None
):
 """
 스트리밍 방식 답변 (대시보드 실시간 표시용)
 """
 pipeline = get_pipeline()
```

```
 def generate():
 for chunk in pipeline.ask_stream(
 query=query,
 model_code=model_code,
 defect_type=defect_type
```

```

):
 yield f"data: {chunk}\n\n"
 yield "data: [DONE]\n\n"

 return StreamingResponse(
 generate(),
 media_type="text/event-stream"
)

@router.post("/index/report")
async def index_report(
 text: str,
 model_code: str,
 defect_type: Optional[str] = None,
 doc_date: Optional[str] = None,
 doc_type: str = "report"
):
 """진단 보고서를 RAG 인덱스에 추가"""
 pipeline = get_pipeline()
 count = pipeline.index_report(
 text=text,
 metadata={
 "model_code": model_code,
 "defect_type": defect_type,
 "doc_date": doc_date,
 "doc_type": doc_type
 }
)
 return {"indexed_chunks": count, "status": "success"}

```

## STEP 10: 데이터 수집 스크립트

```

scripts/ingest_data.py
"""
기존 진단 데이터를 RAG 인덱스에 수집하는 스크립트

```

```

"""

import os
import sys
import pandas as pd
from pathlib import Path
sys.path.append(str(Path(__file__).parent.parent))

from backend.rag.pipeline import SmartphoneRAGPipeline
from dotenv import load_dotenv

load_dotenv()

def ingest_csv_defect_data(pipeline, csv_path: str):
 """CSV 불량 데이터 수집"""
 print(f"\n📁 CSV 수집 중: {csv_path}")
 df = pd.read_csv(csv_path)

 # 모델별로 그룹화해서 수집
 for model_code, group in df.groupby("model_code"):
 rows = group.to_dict("records")
 count = pipeline.index_csv_data(
 rows=rows,
 metadata={
 "model_code": model_code,
 "doc_type": "defect_data",
 "source_file": csv_path
 }
)
 print(f"✅ {model_code}: {count}개 청크 저장")

def ingest_text_reports(pipeline, reports_dir: str):
 """텍스트 분석 보고서 수집"""
 print(f"\n📁 보고서 수집 중: {reports_dir}")

```

```

reports_path = Path(reports_dir)

for report_file in reports_path.glob("*.txt"):
 # 파일명에서 메타데이터 추출
 # 예: SM-S928_HW_BATTERY_2026-04.txt
 parts = report_file.stem.split("_")
 model_code = parts[0] if len(parts) > 0 else None
 defect_type = "_" .join(parts[1:3]) if len(parts) > 2 else None

 text = report_file.read_text(encoding="utf-8")
 count = pipeline.index_report(
 text=text,
 metadata={
 "model_code": model_code,
 "defect_type": defect_type,
 "doc_type": "report",
 "source_file": str(report_file)
 }
)
 print(f" 🟢 {report_file.name}: {count}개 청크 저장")

```

def main():

```

print(" 🚀 RAG 데이터 수집 시작")
print("=" * 50)

```

```

pipeline = SmartphoneRAGPipeline(
 db_url=os.getenv("DATABASE_URL"),
 anthropic_api_key=os.getenv("ANTHROPIC_API_KEY")
)

```

# CSV 불량 데이터 수집

```

if Path("data/processed/defects_2026.csv").exists():
 ingest_csv_defect_data(
 pipeline,
 "data/processed/defects_2026.csv"
)

```

```

)

 # 텍스트 보고서 수집
 if Path("data/reports").exists():
 ingest_text_reports(pipeline, "data/reports")

 print("\n✅ 수집 완료!")

if __name__ == "__main__":
 main()

```

## STEP 11: RAGAS 평가

```

tests/test_rag.py

import os
import pytest
from backend.rag.pipeline import SmartphoneRAGPipeline
from dotenv import load_dotenv

load_dotenv()

@pytest.fixture(scope="module")
def pipeline():
 return SmartphoneRAGPipeline(
 db_url=os.getenv("DATABASE_URL"),
 anthropic_api_key=os.getenv("ANTHROPIC_API_KEY")
)

테스트 케이스 (실제 질문 + 기대 답변)
TEST_CASES = [
 {
 "question": "SM-S928의 배터리 불량 주요 증상은?",
 "expected_keywords": ["배터리", "충전", "SM-S928"]
 }
]

```

```

 },
 {
 "question": "HW_DISPLAY 불량률의 심각도 분포는?",
 "expected_keywords": ["화면", "디스플레이", "심각도"]
 },
 {
 "question": "가장 불량률이 높은 모델은?",
 "expected_keywords": ["불량률", "모델"]
 }
]

```

```

def test_retrieval_returns_results(pipeline):
 """검색이 결과를 반환하는지 확인"""
 candidates = pipeline.retriever.retrieve(
 "배터리 불량", top_k=5
)
 assert len(candidates) > 0, "검색 결과가 없습니다"
 assert "chunk_text" in candidates[0]

```

```

def test_answer_contains_keywords(pipeline):
 """답변이 관련 키워드를 포함하는지 확인"""
 for tc in TEST_CASES:
 result = pipeline.ask(tc["question"])
 answer = result["answer"].lower()

 matched = sum(
 1 for kw in tc["expected_keywords"]
 if kw.lower() in answer
)
 assert matched >= 1, W
 f"질문 '{tc['question']}'의 답변에 키워드 없음: {answer[:200]}"

```

```

def test_structured_output_format(pipeline):

```

```

"""구조화 출력 형식 검증"""
result = pipeline.ask(
 "SM-S928 배터리 불량 요약",
 structured=True
)
assert "summary" in result
assert "key_findings" in result
assert isinstance(result["key_findings"], list)

def test_filter_reduces_results(pipeline):
 """필터 적용 시 결과가 줄어드는지 확인"""
 all_results = pipeline.retriever.retrieve(
 "불량", top_k=20
)
 filtered_results = pipeline.retriever.retrieve(
 "불량", top_k=20,
 model_code="SM-S928"
)
 # 필터링된 결과가 더 적거나 같아야 함
 assert len(filtered_results) <= len(all_results)

def test_latency_acceptable(pipeline):
 """응답 시간이 10초 이내인지 확인"""
 import time
 start = time.time()
 pipeline.ask("배터리 불량률은?")
 elapsed = time.time() - start
 assert elapsed < 10, f"응답 시간 초과: {elapsed:.1f}초"

if __name__ == "__main__":
 # 간단한 수동 테스트
 p = SmartphoneRAGPipeline(
 db_url=os.getenv("DATABASE_URL"),

```



```

anthropic_api_key=os.getenv("ANTHROPIC_API_KEY")
)

questions = [
 "SM-S928의 최근 배터리 불량 트렌드는?",
 "심각도 5인 불량률의 주요 원인 분석해줘",
 "이번 달 모델별 불량률 비교해줘"
]

for q in questions:
 print(f"\n질문: {q}")
 print("-" * 50)
 result = p.ask(q)
 print(f"답변: {result['answer'][:300]}...")
 print(f"검색: {result['retrieved_count']}건 → {result['final_context_count']}건")
 print(f"지연: {result['latency_ms']}ms")

```

## STEP 12: 실행 및 테스트

```

1. DB 스키마 생성
psql $DATABASE_URL -f backend/db/schema.sql

2. 데이터 수집
python scripts/ingest_data.py

3. 서버 시작
uvicorn backend.main:app --reload --port 8000

4. API 테스트
curl -X POST "http://localhost:8000/api/v1/rag/ask" \
 -H "Content-Type: application/json" \
 -d '{
 "query": "SM-S928 배터리 불량 원인 분석해줘",
 "model_code": "SM-S928",
 "defect_type": "HW_BATTERY",

```

```
"structured": false
}
```

#### # 5. 테스트 실행

```
pytest tests/test_rag.py -v
```

#### # 스트리밍 테스트

```
curl -N "http://localhost:8000/api/v1/rag/ask/streamW
?query=배터리불량원인&model_code=SM-S928"
```

### 성능 기준 (프로젝트 목표)

하이브리드 검색은 밀집 검색만 사용하는 것보다 20~40% 높은 검색 재현율을 보였습니다. 크로스 인코더 리랭킹은 18~42% 정밀도 향상을 제공합니다.

#### 달성 목표:

검색 재현율	> 80%	
답변 정확도	> 85%	
응답 지연	< 5초 (P95)	
토큰 비용	< \$0.01 / 질문	

#### 단계별 개선:

**Naive RAG** → 기준 (40% 실패율)

+ 하이브리드 → +20~40% 재현율

+ 리랭킹 → +18~42% 정밀도

+ 필터링 → +60% 특화 정확도